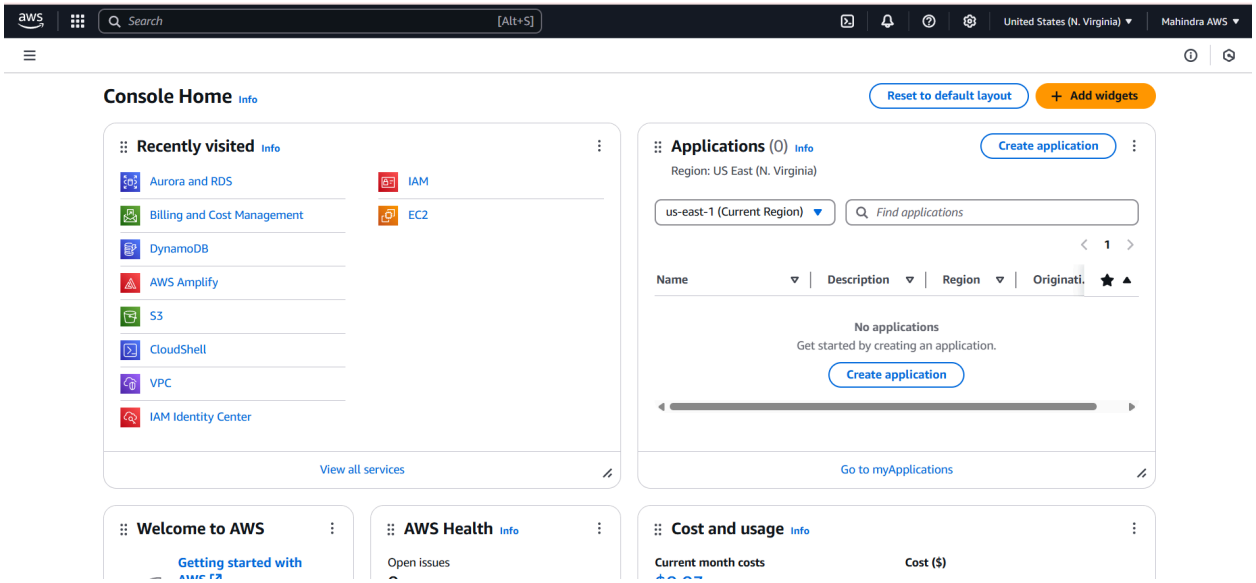
AWS VPC, Route53, and CloudFront Lab

Author : Mahindra Guptha Kotha
Mail ID: [redacted]

Ex : 4.1

CREATE A NEW VPC

Create a VPC with the primary CIDR [redacted]

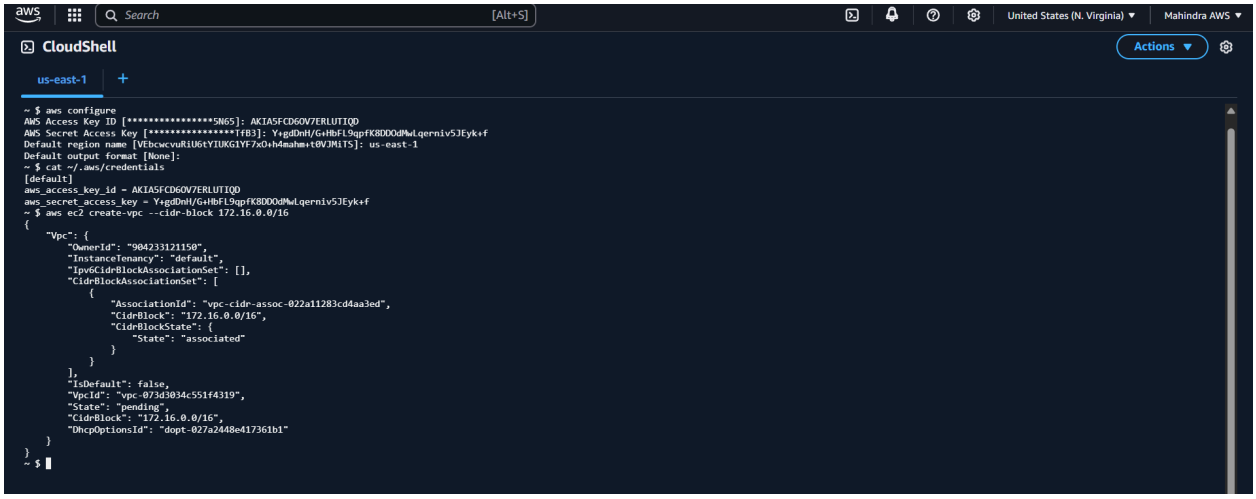


To complete this exercise using the AWS Command-Line Interface, enter the following command:

Before this run

Aws configure and give the newly created user access key details and make sure AAmazonVPCFullaccess policy is add to the user.

```
aws ec2 create-vpc
--cidr-block [redacted]
```



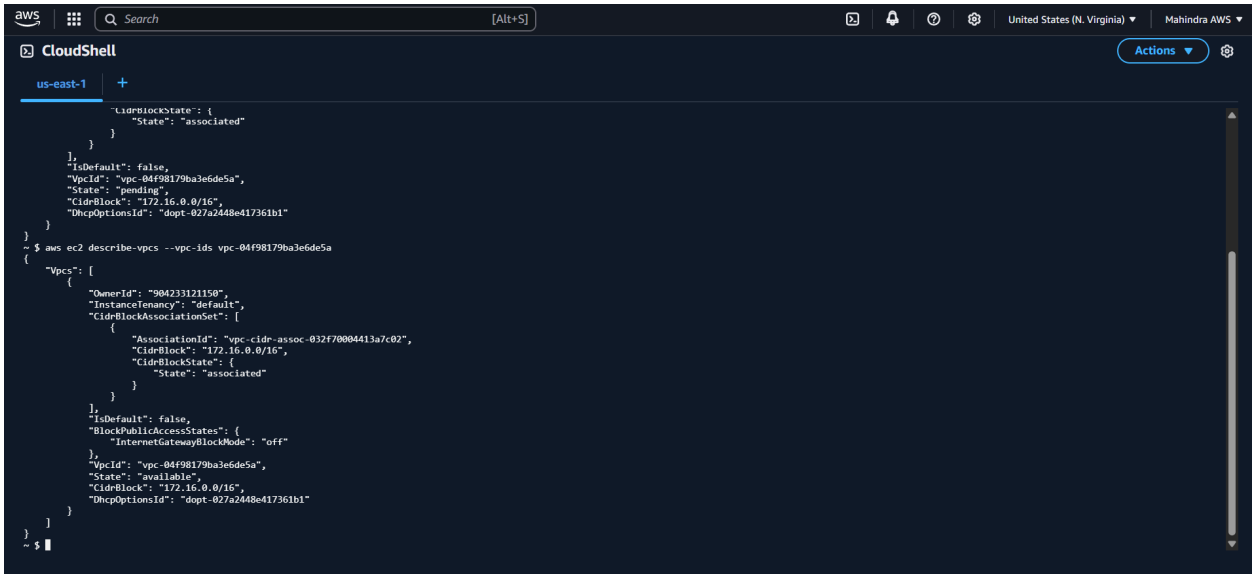
If the command succeeds, you should see output similar to the following:


```
{
    "Vpc": {
        "OwnerId": "[redacted]",
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId": "vpc-cidr-assoc-[redacted]",
                "CidrBlock": "[redacted]",
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "VpcId": "vpc-04f98179ba3e6de5a",
        "State": "pending",
        "CidrBlock": "[redacted]",
        "DhcpOptionsId": "dopt-[redacted]"
    }
}
```


Note that the VPC is initially in the pending state as AWS creates it. To view its state, note the VPC identifier and issue the following command:

aws ec2 describe-vpcs --vpc-ids **vpc-04f98179ba3e6de5a**

You should see output similar to the following:




```
{
   "Vpcs": [
      {
         "OwnerId": "[redacted]",
         "InstanceTenancy": "default",
         "CidrBlockAssociationSet": [
            {
               "AssociationId": "vpc-cidr-assoc-[redacted]",
               "CidrBlock": "[redacted]",
               "CidrBlockState": {
                  "State": "associated"
               }
            }
         ],
         "IsDefault": false,
         "BlockPublicAccessStates": {
            "InternetGatewayBlockMode": "off"
         },
         "VpcId": "vpc-04f98179ba3e6de5a",
         "State": "available",
         "CidrBlock": "[redacted]",
         "DhcpOptionsId": "dopt-[redacted]"
      }
   ]
}
```


Your new VPC is now ready to use. Keep it handy, as you'll be using it in subsequent exercises. You can delete an unneeded VPC using code like this:

```
aws ec2 delete-vpc --vpc-id vpc-a01106c2
```

EXERCISE 4.2

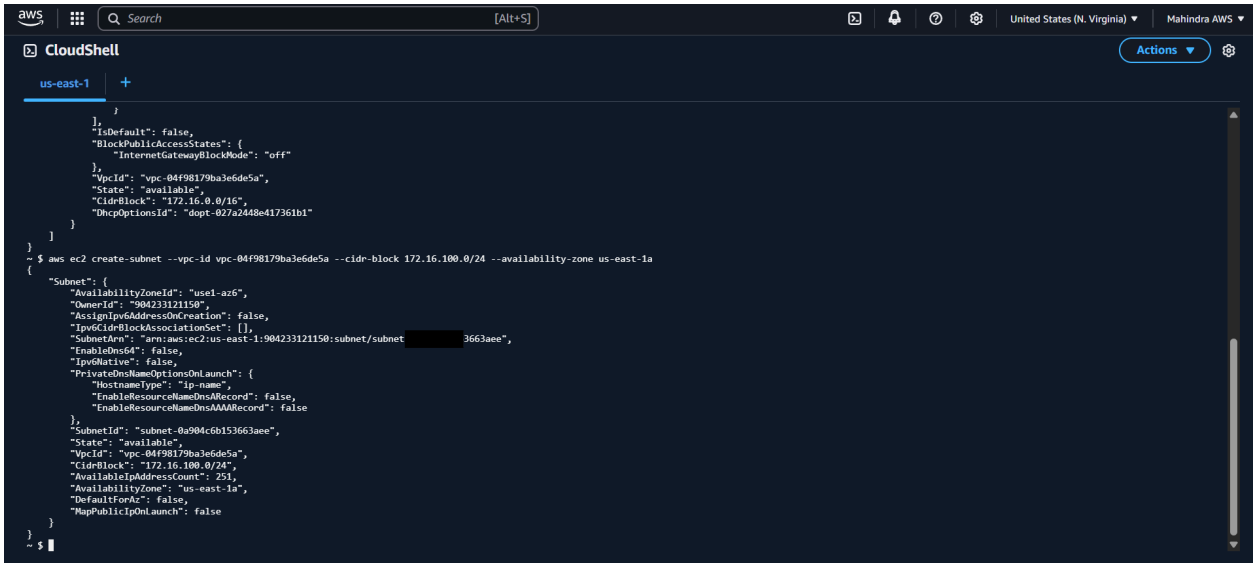
CREATE A NEW SUBNET

Create a subnet in the VPC you created earlier. Choose an availability zone and assign the CIDR block [redacted]

To complete this using the following AWS Command-Line Interface commands, you'll need the resource ID of the VPC:

```
aws ec2 create-subnet
--vpc-id [vpc-id]
--cidr-block [redacted]
--availability-zone us-east-1a
```

Assuming you're running the command from the matching region, you should see output similar to the following:



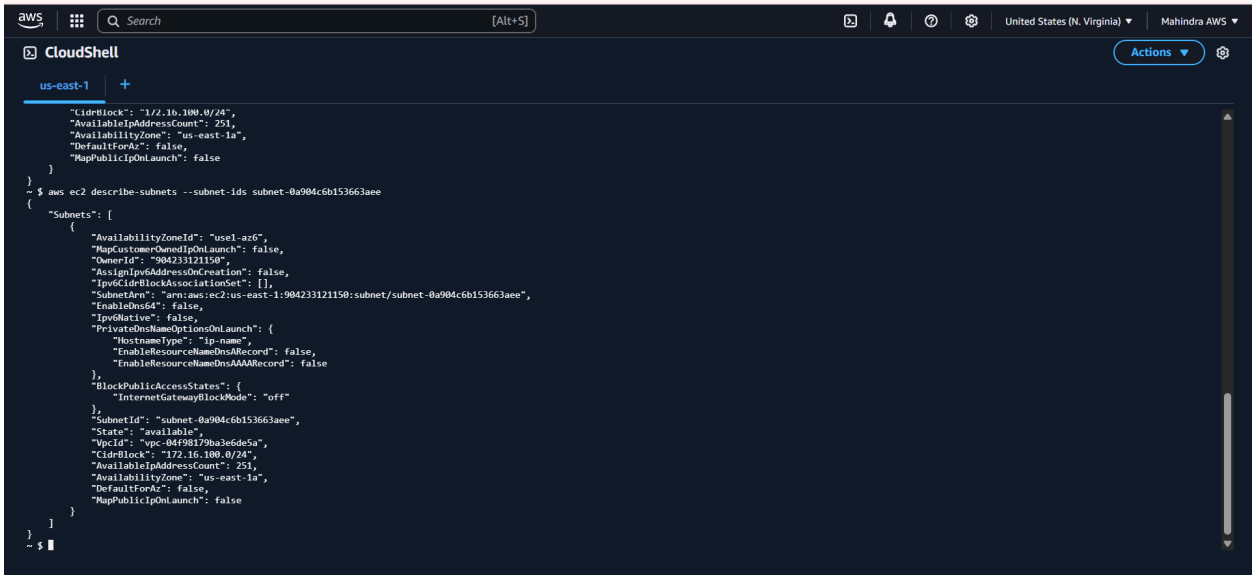
This is how it looks


```
{
    "Subnet": {
        "AvailabilityZoneId": "use1-az6",
        "OwnerId": "[redacted]",
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "SubnetArn": "arn:aws:ec2:us-east-1:[redacted] ubnet/subnet-0a904c6b153663aee",
        "EnableDns64": false,
        "Ipv6Native": false,
        "PrivateDnsNameOptionsOnLaunch": {
            "HostnameType": "ip-name",
```

```
            "EnableResourceNameDnsARecord": false,
            "EnableResourceNameDnsAAAARecord": false
        },
        "SubnetId": "subnet-[redacted]ee",
        "State": "available",
        "VpcId": "vpc-04f98179ba3e6de5a",
        "CidrBlock": "[redacted],
        "AvailableIpAddressCount": 251,
        "AvailabilityZone": "us-east-1a",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false
    }
}
```


Wait a few moments, and then check the status of the subnet as follows:

aws ec2 describe-subnets --subnet-ids subnet-[redacted] e



You should see the subnet in an available state, like so:


```
{
    "Subnets": [
        {
            "AvailabilityZoneId": "use1-az6",
            "MapCustomerOwnedIpOnLaunch": false,
            "OwnerId": "[redacted]",
            "AssignIpv6AddressOnCreation": false,
            "Ipv6CidrBlockAssociationSet": [],
            "SubnetArn":
"arn:aws:ec2:us-east-1:[redacted]subnet/subnet-0a904c6b153663aee",
            "EnableDns64": false,
```

```
            "Ipv6Native": false,
            "PrivateDnsNameOptionsOnLaunch": {
                "HostnameType": "ip-name",
                "EnableResourceNameDnsARecord": false,
                "EnableResourceNameDnsAAAARecord": false
            },
            "BlockPublicAccessStates": {
                "InternetGatewayBlockMode": "off"
            },
            "SubnetId": "subnet-[redacted]ee",
            "State": "available",
            "VpcId": "vpc-04f98179ba3e6de5a",
            "CidrBlock": "[redacted],
            "AvailableIpAddressCount": 251,
            "AvailabilityZone": "us-east-1a",
            "DefaultForAz": false,
            "MapPublicIpOnLaunch": false
        }
    ]
}
```

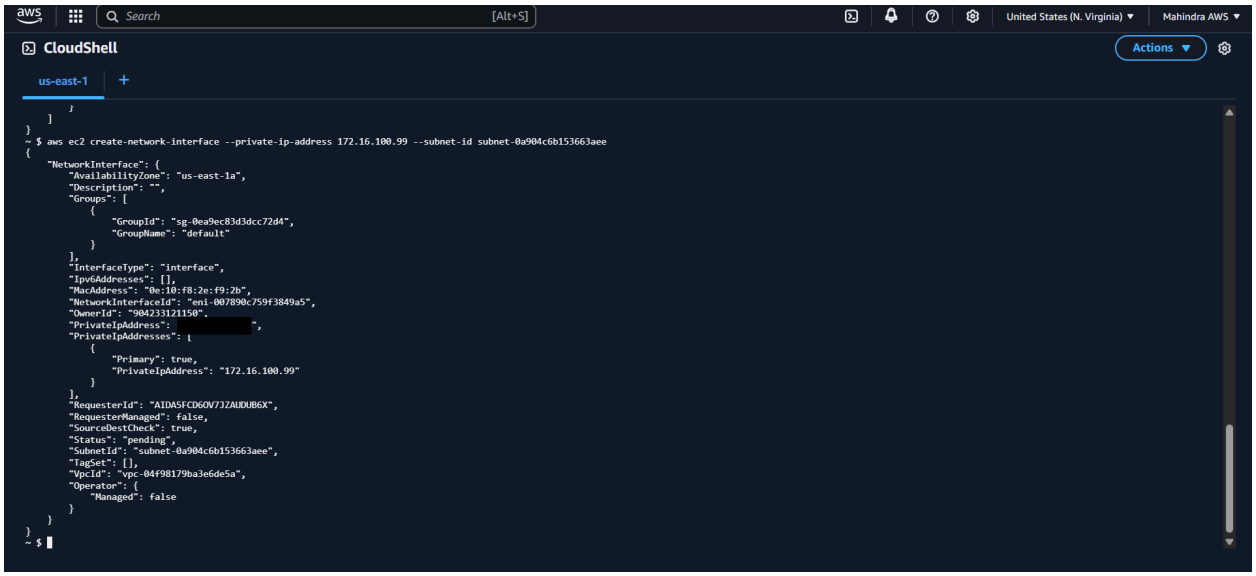

EXERCISE 4.3

CREATE AND ATTACH A PRIMARY ENI

Create an ENI in the subnet of your choice.

Using the subnet ID from the previous exercise, issue the following AWS CLI command:

aws ec2 create-network-interface --private-ip-address [redacted] --subnet-id subnet-0a904c6b153663aee

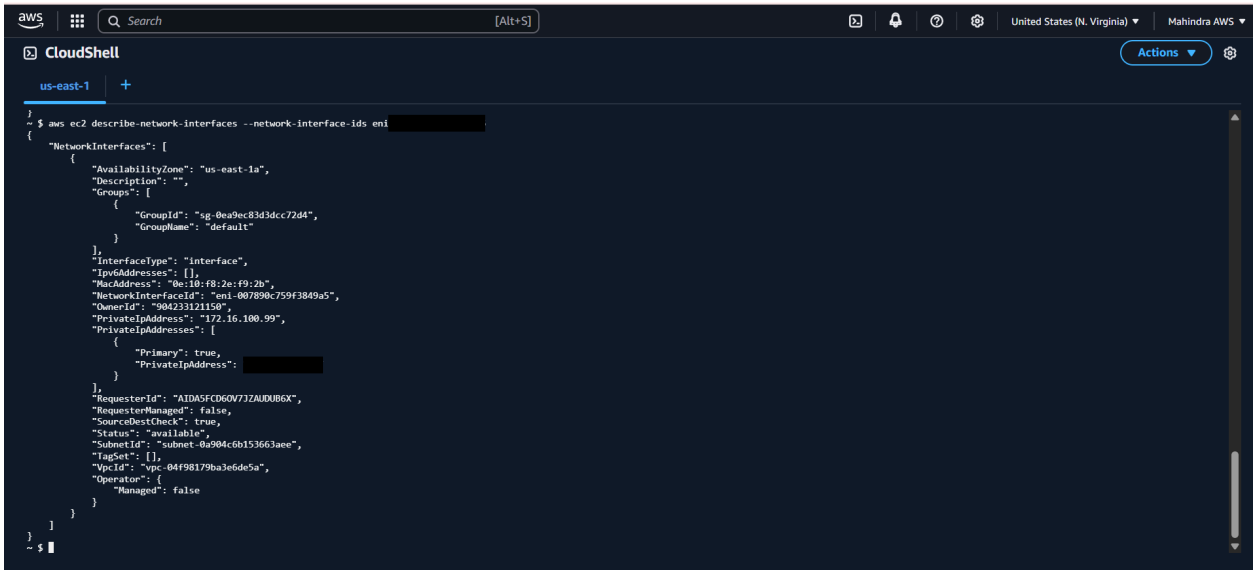


You should see output similar to the following:


```
{
    "NetworkInterface": {
        "AvailabilityZone": "us-east-1a",
        "Description": "",
        "Groups": [
            {
                "GroupId": "sg-0ea9ec83d3dcc72d4",
                "GroupName": "default"
            }
        ],
        "InterfaceType": "interface",
        "Ipv6Addresses": [],
        "MacAddress": "0e:10:f8:2e:f9:2b",
        "NetworkInterfaceId": "eni-0 [redacted]
        "OwnerId": "[redacted]",
        "PrivateIpAddress": "[redacted],
        "PrivateIpAddresses": [
            {
                "Primary": true,
                "PrivateIpAddress": "[redacted]
            }
        ],
        "RequesterId": "AIDA5FCD6OV7JZAUDUB6X",
        "RequesterManaged": false,
        "SourceDestCheck": true,
        "Status": "pending",
        "SubnetId": "subnet-[redacted] ee",
        "TagSet": [],
        "VpcId": "vpc-04f98179ba3e6de5a",
        "Operator": {
            "Managed": false
        }
    }
}
```


After a few moments, verify the status of the network interface as follows, using the NetworkInterfaceId from the preceding output:

```
aws ec2 describe-network-interfaces --network-interface-ids eni-[redacted]
```

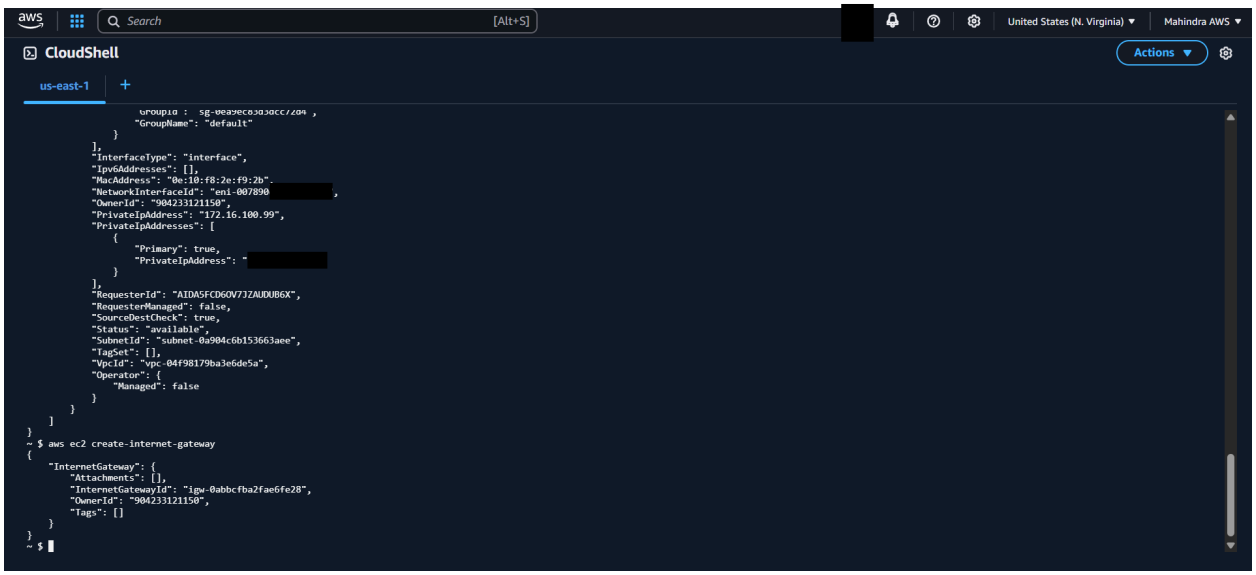


EXERCISE 4.4

CREATE AN INTERNET GATEWAY AND DEFAULT ROUTE

In this exercise, you'll create an Internet gateway and attach it to the VPC you used in the previous exercises.

Create an Internet gateway using the following command:
aws ec2 create-internet-gateway

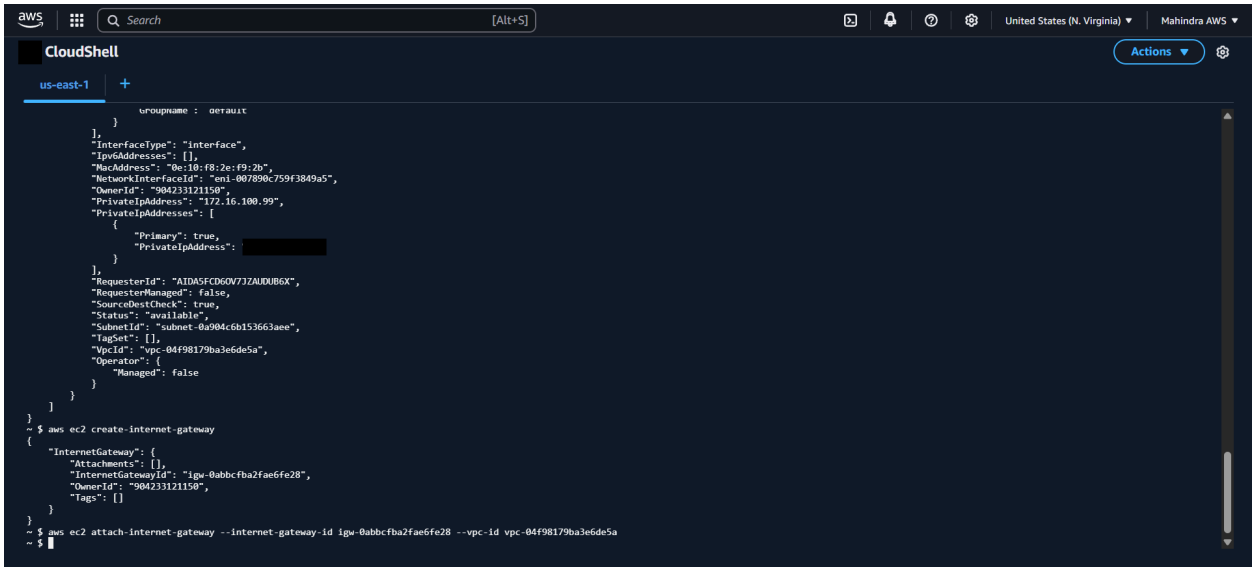


You should see the following output:

```
{
    "InternetGateway": {
        "Attachments": [],
        "InternetGatewayId": "igw-0abbcfba2fae6fe28",
        "OwnerId": "[redacted]",
        "Tags": []
    }
}
```

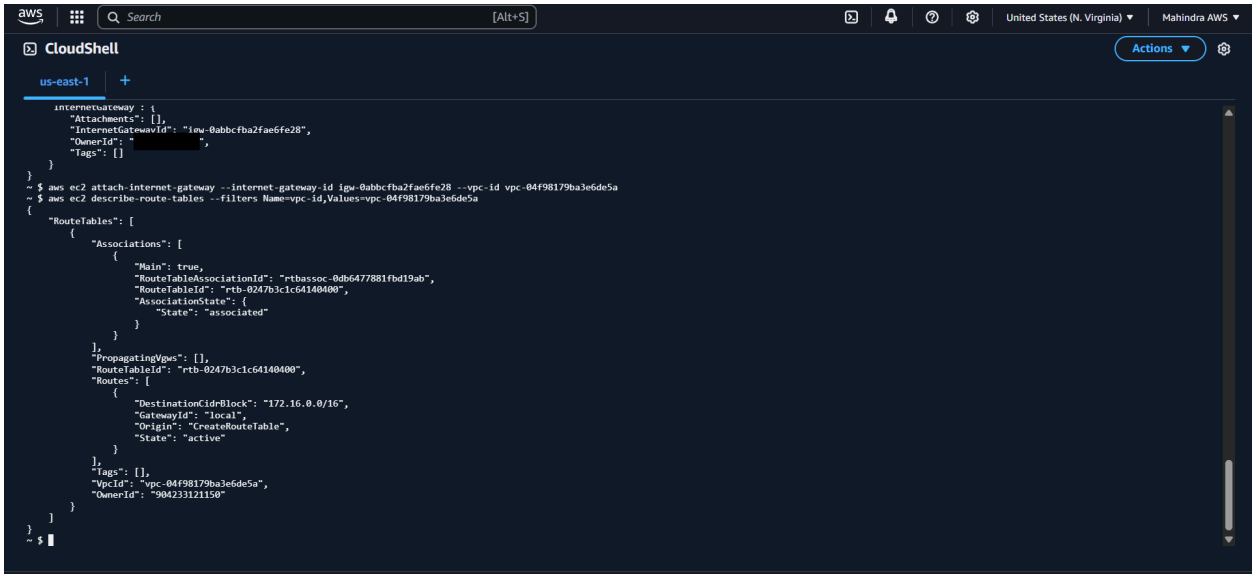
Attach the Internet gateway to the VPC you used in the previous exercises using the following command (a successful execution will generate no output):

```
aws ec2 attach-internet-gateway --internet-gateway-id igw-0abbcfba2fae6fe28 --vpc-id vpc-04f98179ba3e6de5a
```



Retrieve the route table ID of the main route table for the VPC:

```
aws ec2 describe-route-tables --filters Name=vpc-id,Values=vpc-04f98179ba3e6de5a
```



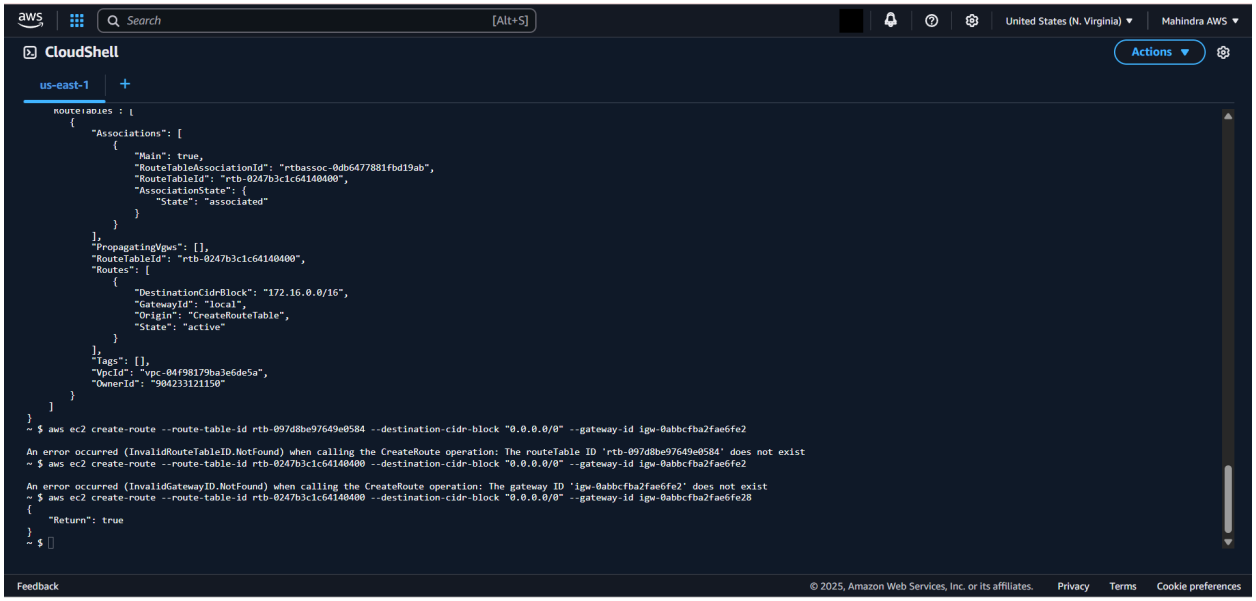
You should see something like the following:


```
{
    "RouteTables": [
        {
            "Associations": [
                {
                    "Main": true,
                    "RouteTableAssociationId": "rtbassoc-0db6477881fbd19ab",
                    "RouteTableId": "rtb- [redacted],
                    "AssociationState": {
                        "State": "associated"
                    }
                }
            ],
            "PropagatingVgws": [],
            "RouteTableId": "rtb- [redacted],
            "Routes": [
                {
                    "DestinationCidrBlock": "[redacted],
                    "GatewayId": "local",
                    "Origin": "CreateRouteTable",
                    "State": "active"
                }
            ],
            "Tags": [],
            "VpcId": "vpc-04f98179ba3e6de5a",
            "OwnerId": "[redacted]"
        }
```

]
}

To create a default route in the main route table, issue the following command:

aws ec2 create-route --route-table-id rtb- [redacted] --destination-cidr-block "0.0.0.0/0" --gateway-id igw-0abbcfba2fae6fe28



If the command succeeds, you should see the following output:

{
"Return": true
}
Confirm the presence of the route by rerunning the command from step 3.

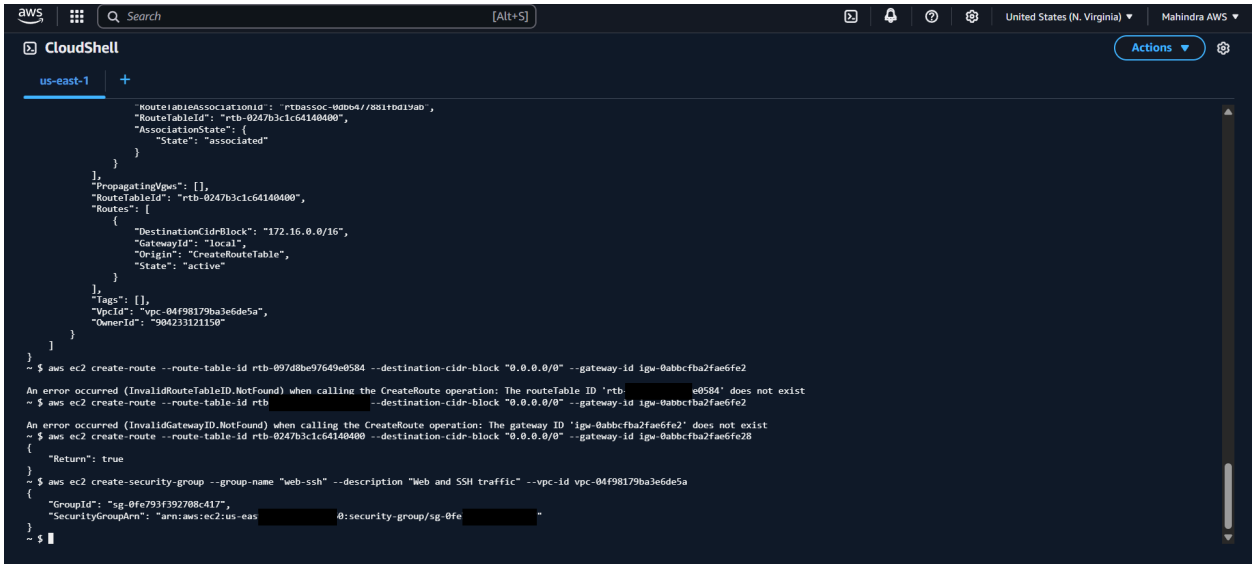
EXERCISE 4.5

CREATE A CUSTOM SECURITY GROUP

In this exercise, you'll create a new security group that allows SSH, HTTP, and HTTPS access from any IP address.

Create a security group named web-ssh in the VPC you created previously:

aws ec2 create-security-group --group-name "web-ssh" --description "Web and SSH traffic" --vpc-id vpc-04f98179ba3e6de5a



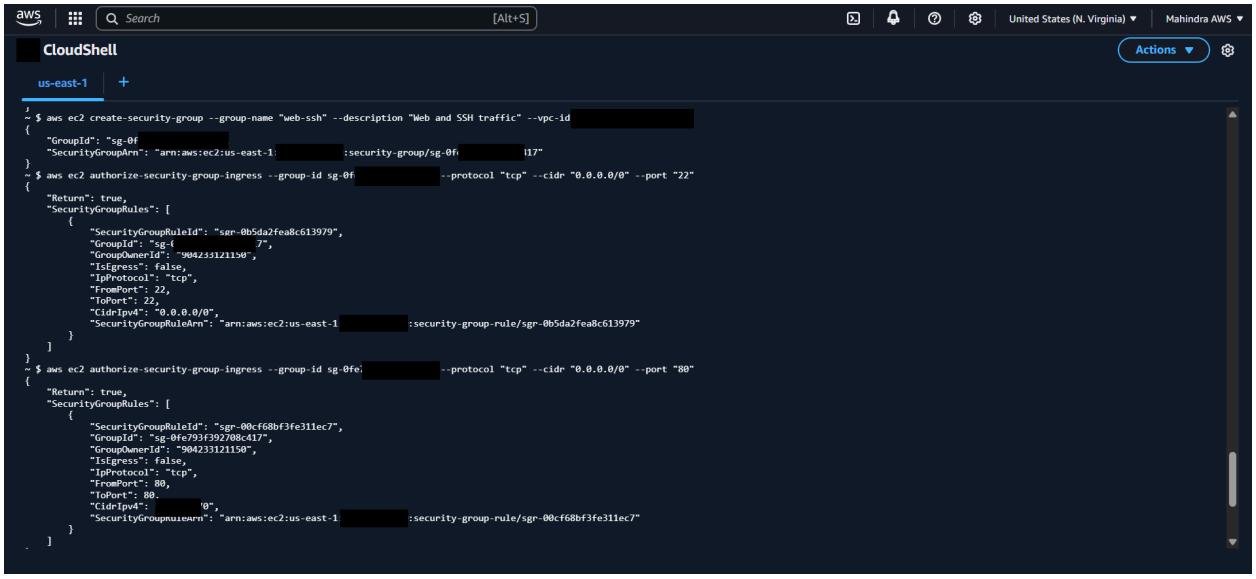
Make a note of the security group ID in the output:

{
"GroupId": "sg-0fe[redacted]",
"SecurityGroupArn":
"arn:aws:ec2:us-east-1:[redacted]:security-group/sg-0fe793f392708c417"
}

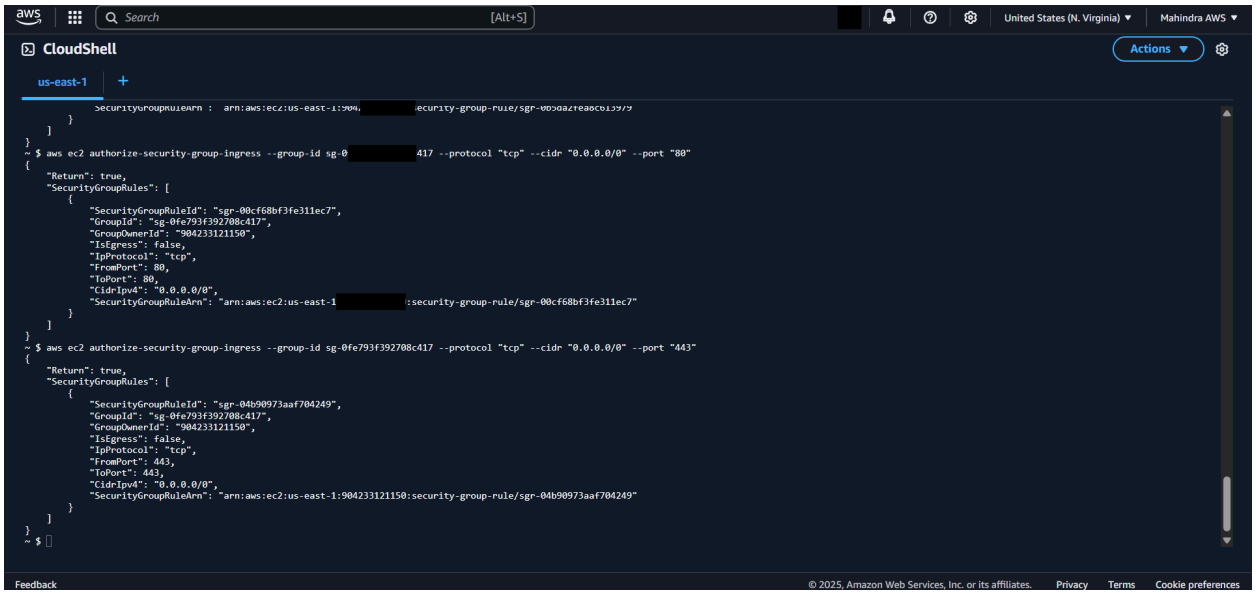
Using the security group ID, create three rules to allow SSH, HTTP, and HTTPS access from any IP address.

aws ec2 authorize-security-group-ingress --group-id sg-0fe[redacted] --protocol "tcp" --cidr "0.0.0.0/0" --port "22"

aws ec2 authorize-security-group-ingress --group-id sg-0fe[redacted] --protocol "tcp" --cidr "0.0.0.0/0" --port "80"

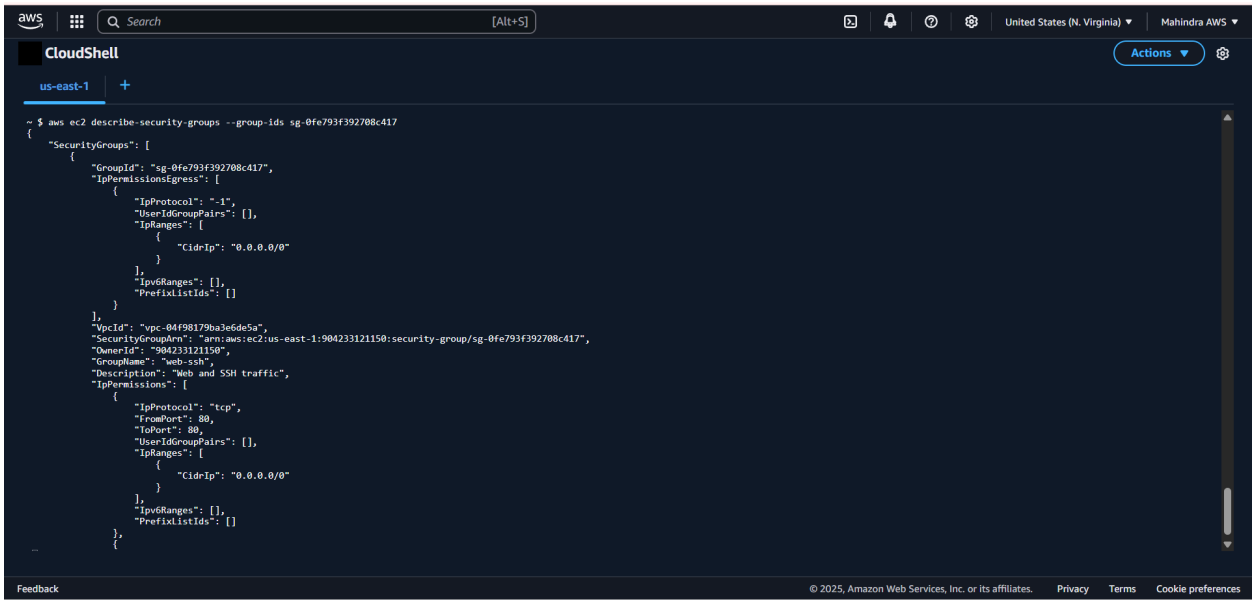


aws ec2 authorize-security-group-ingress --group-id sg-0fe [redacted] --protocol "tcp" --cidr "0.0.0.0/0" --port "443"



Verify the rules by viewing the security group:

aws ec2 describe-security-groups -group-ids sg-0fe [redacted]

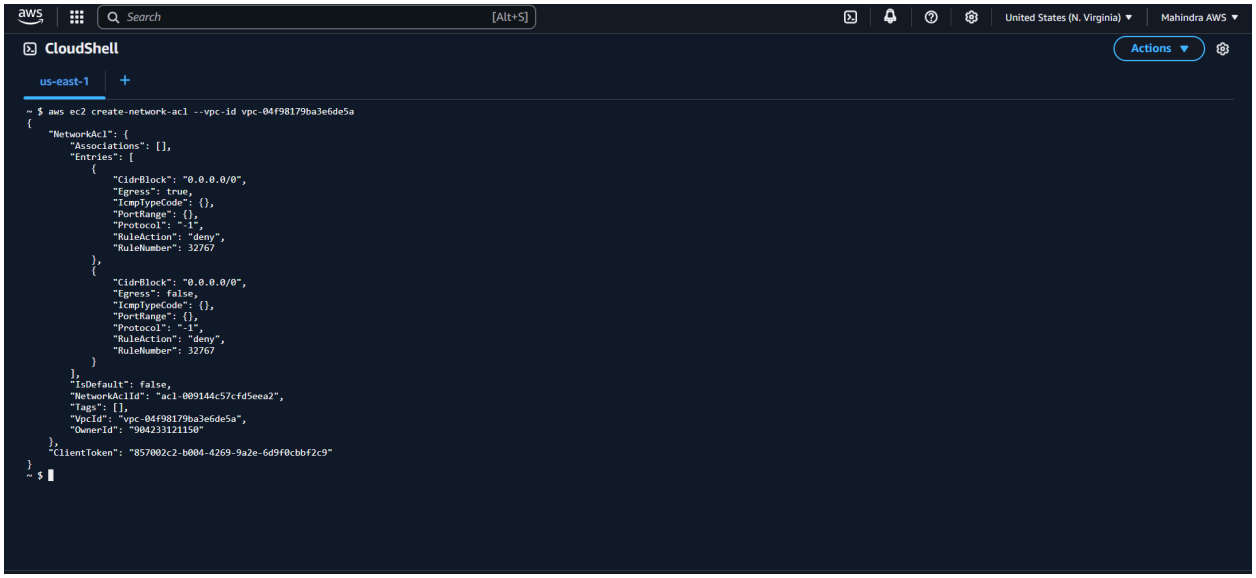


EXERCISE 4.6

Create an Inbound Rule to Allow Remote Access from Any IP Address

NACL rule order matters! Create a new NACL and attach it to the subnet you used in Exercise 4.3.

Create a new network ACL using the following command:
aws ec2 create-network-acl --vpc-id vpc-04f98179ba3e6de5a



Note the network ACL ID in the output, which should look something like the following:

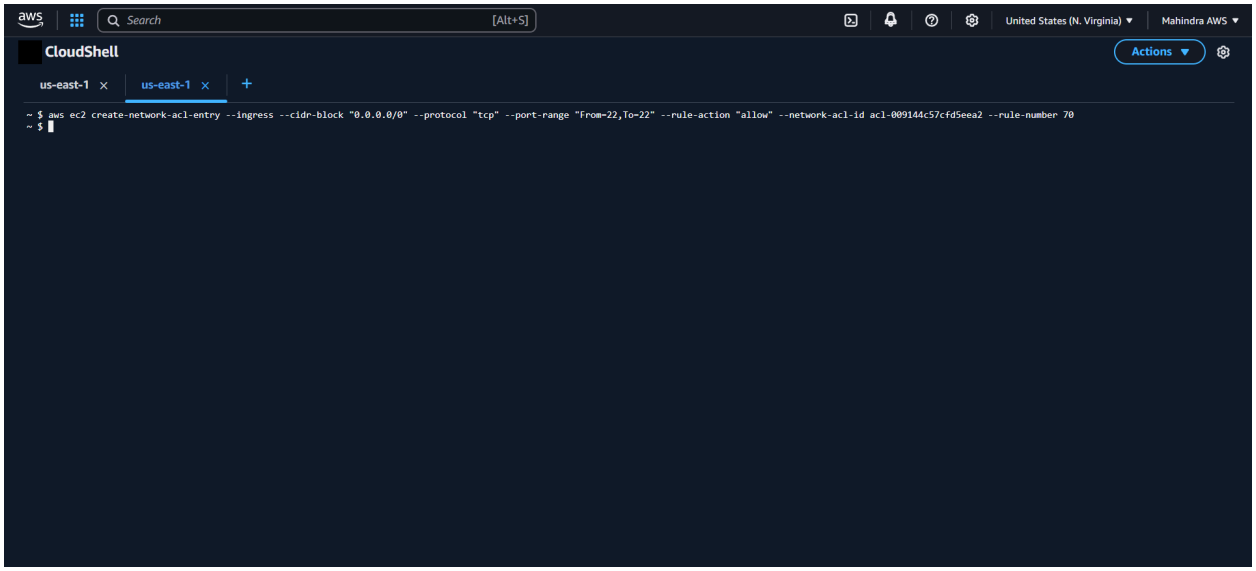

```
{
  "NetworkAcl": {
    "Associations": [],
    "Entries": [
      {
        "CidrBlock": "0.0.0.0/0",
        "Egress": true,
        "IcmpTypeCode": {},
        "PortRange": {},
        "Protocol": "-1",
        "RuleAction": "deny",
        "RuleNumber": 32767
      },
      {
        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "IcmpTypeCode": {},
        "PortRange": {},
        "Protocol": "-1",
        "RuleAction": "deny",
        "RuleNumber": 32767
      }
    ],
    "IsDefault": false,
    "NetworkAclId": "acl-009144c57cfd5eea2",
```

```
        "Tags": [],
        "VpcId": "vpc-04f98179ba3e6de5a",
        "OwnerId": [redacted]
    },
    "ClientToken": "857002c2- [redacted] -6d9f0cbbf2c9"
}
```


Notice that the network ACL includes the default ingress and egress rules to deny all traffic. Even though the rule number is 32767, it shows up in the AWS Management Console as an asterisk.

Create an inbound rule entry to allow SSH (TCP port 22) access from any IP address. We'll use the rule number 70.

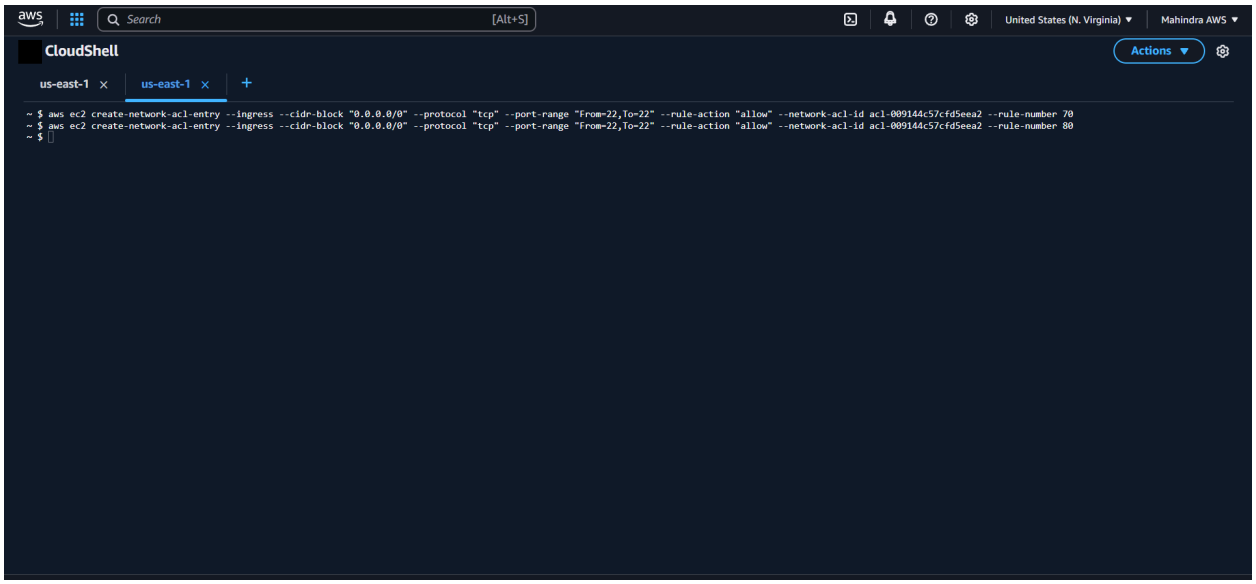
```
aws ec2 create-network-acl-entry --ingress --cidr-block "0.0.0.0/0" --protocol "tcp" --port-range "From=22,To=22" --rule-action "allow" --network-acl-id acl-009144c57cfd5eea2 --rule-number 70
```



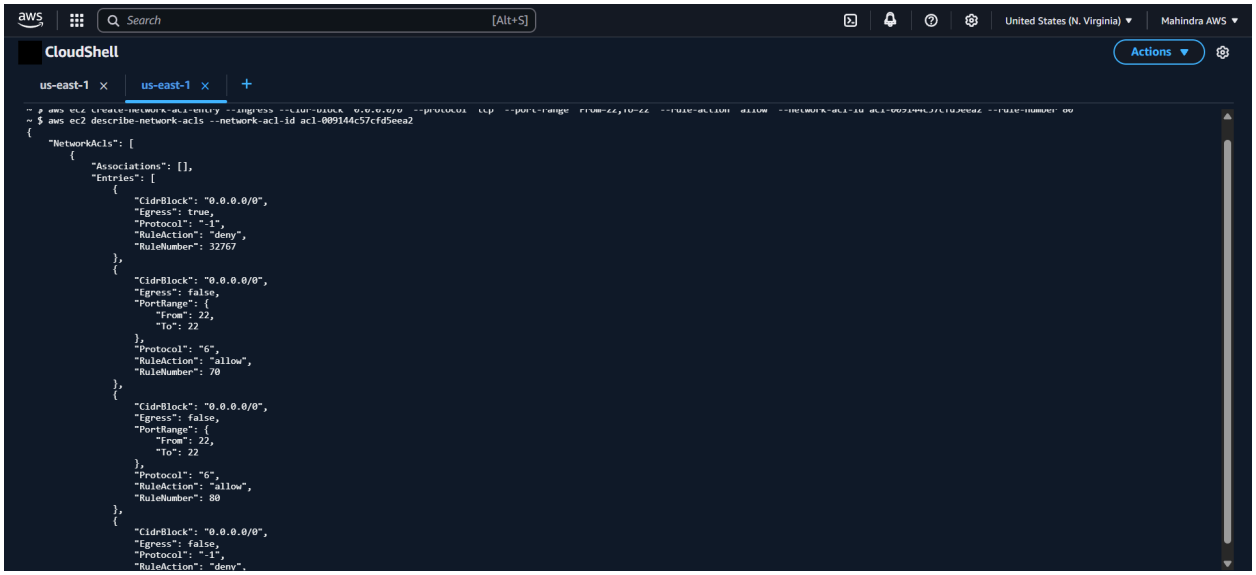
The `From=` and `To=` values after the `--port-range` flag refer to the range of ports to allow, *not* to the source port number. In this rule, we just want to allow traffic to TCP port 22.

Modify the command from step 2 to create rule number 80 that allows RDP (TCP port 3389) access from your public IP address (use https://ifconfig.co to find it if you don't already know it). Use a /32 prefix length.

aws ec2 create-network-acl-entry --ingress --cidr-block "0.0.0.0/0" --protocol "tcp" --port-range "From=22,To=22" --rule-action "allow" --network-acl-id acl-009144c57cfd5eea2 --rule-number 80



Use the following command to verify that the rules made it into your NACL:
aws ec2 describe-network-acls --network-acl-id acl-009144c57cfd5eea2



EXERCISE 4.7

Allocate and Use an Elastic IP Address

Allocate an elastic IP address and associate it with the instance you created earlier.

Allocate an EIP using the following command:

```
aws ec2 allocate-address
```

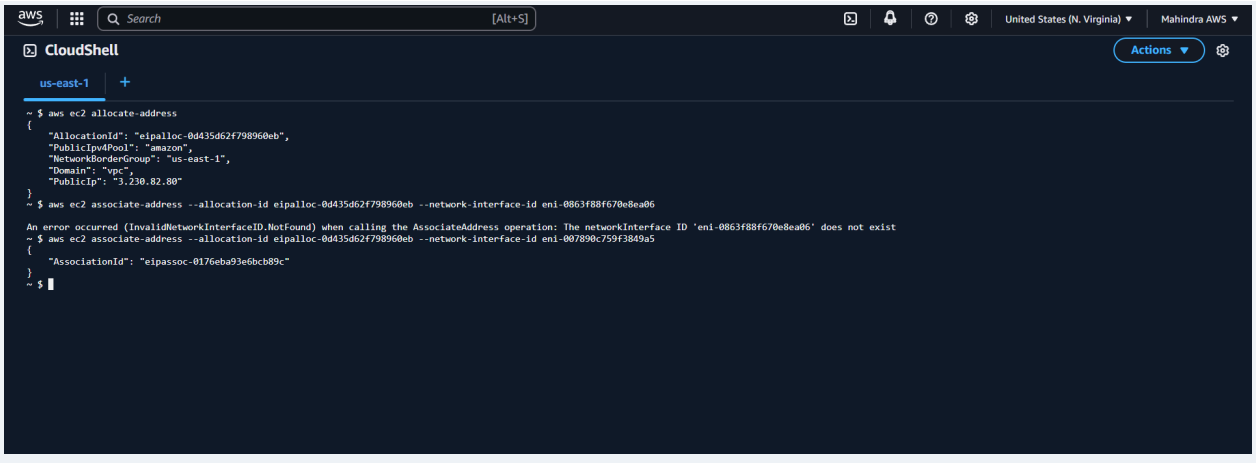


You should see a public IP address and allocation ID in the output:

```
{
    "AllocationId": "eipalloc-[redacted]b",
    "PublicIpv4Pool": "amazon",
    "NetworkBorderGroup": "us-east-1",
    "Domain": "vpc",
    "PublicIp": [redacted]
}
```

Associate the EIP to the elastic network interface you created earlier:

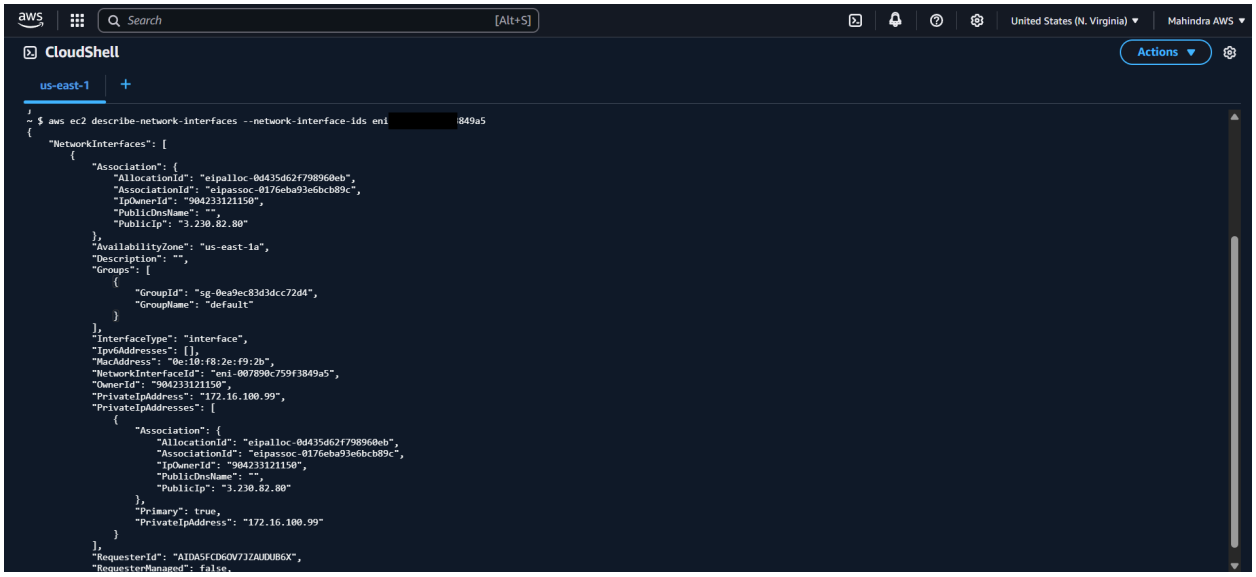
aws ec2 associate-address --allocation-id eipalloc-0[redacted] --network-interface-id eni-[redacted]




```
{
    "AssociationId": "eipassoc-0176eba93e6bcb89c"
}
```

Verify that the EIP is associated with the elastic network interface:

aws ec2 describe-network-interfaces --network-interface-ids eni-[redacted]



You should see the public IP address in the output:


```
{
    "NetworkInterfaces": [
        {
            "Association": {
                "AllocationId": "eipalloc-[redacted]b",
                "AssociationId": "eipassoc-0176eba93e6bcb89c",
                "IpOwnerId": [redacted],
                "PublicDnsName": "",
                "PublicIp": [redacted]
            },
            "AvailabilityZone": "us-east-1a",
            "Description": "",
            "Groups": [
                {
                    "GroupId": "sg-0ea9ec83d3dcc72d4",
                    "GroupName": "default"
                }
            ],
            "InterfaceType": "interface",
            "Ipv6Addresses": [],
            "MacAddress": "0e:10:f8:2e:f9:2b",
            "NetworkInterfaceId": "eni [redacted],
            "OwnerId": [redacted],
            "PrivateIpAddress": "[redacted],
            "PrivateIpAddresses": [
```

```
                {
                    "Association": {
                        "AllocationId":
"eipalloc [redacted]b",
                        "AssociationId":
"eipassoc-0176eba93e6bcb89c",
                        "IpOwnerId": [redacted],
                        "PublicDnsName": "",
                        "PublicIp": [redacted]
                    },
                    "Primary": true,
                    "PrivateIpAddress": "[redacted]
                }
            ],
            "RequesterId": "AIDA5FCD6OV7JZAUDUB6X",
            "RequesterManaged": false,
            "SourceDestCheck": true,
            "Status": "available",
            "SubnetId": "subnet [redacted]ee",
            "TagSet": [],
            "VpcId": "vpc-04f98179ba3e6de5a",
            "Operator": {
                "Managed": false
            }
        }
    ]
}
(END)
```

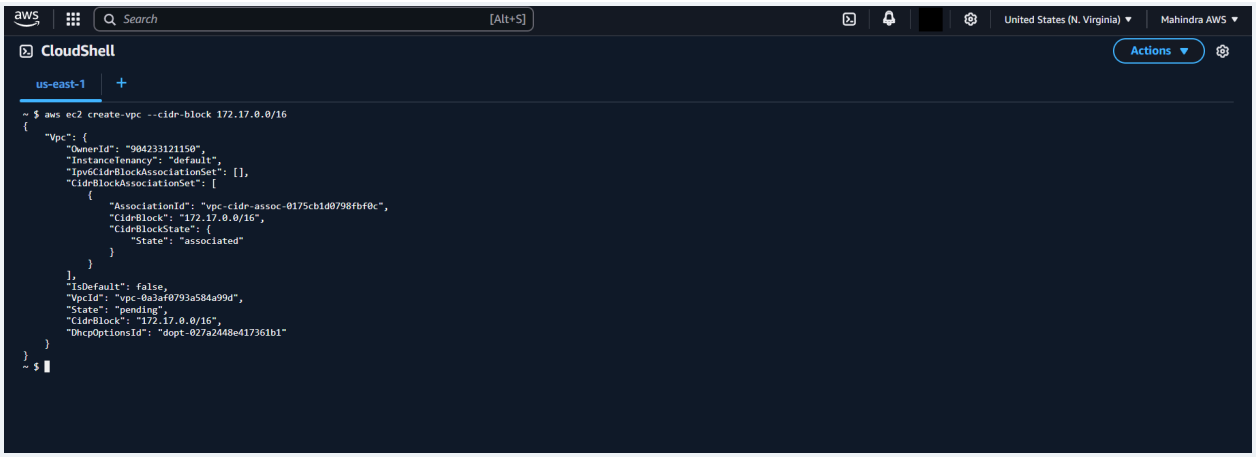

EXERCISE 4.8

Create a Transit Gateway

In this exercise, you'll create another VPC and subnet. You'll then create a transit gateway and attach it to both VPCs. Finally, you'll configure the transit gateway to route between the VPC subnets.

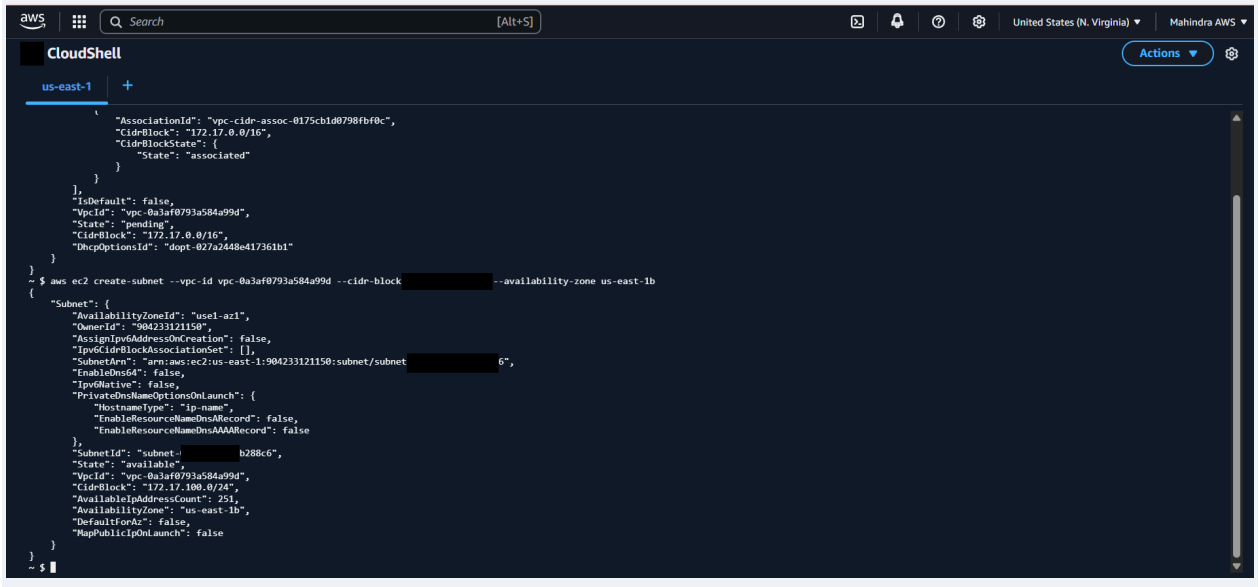
Create a new VPC and subnet (edit the command to match your preferred region and availability zone):

```
 aws ec2 create-vpc --cidr-block [redacted]
```




```
Here is the result:
{
    "Vpc": {
        "OwnerId": [redacted],
        "InstanceTenancy": "default",
        "Ipv6CidrBlockAssociationSet": [],
        "CidrBlockAssociationSet": [
            {
                "AssociationId":
"vpc-cidr-assoc-0175cb1d0798fbf0c",
                "CidrBlock": "[redacted],
                "CidrBlockState": {
                    "State": "associated"
                }
            }
        ],
        "IsDefault": false,
        "VpcId": "vpc-0a3a[redacted]",
        "State": "pending",
        "CidrBlock": "[redacted],
        "DhcpOptionsId": "dopt-[redacted]
    }
}
```

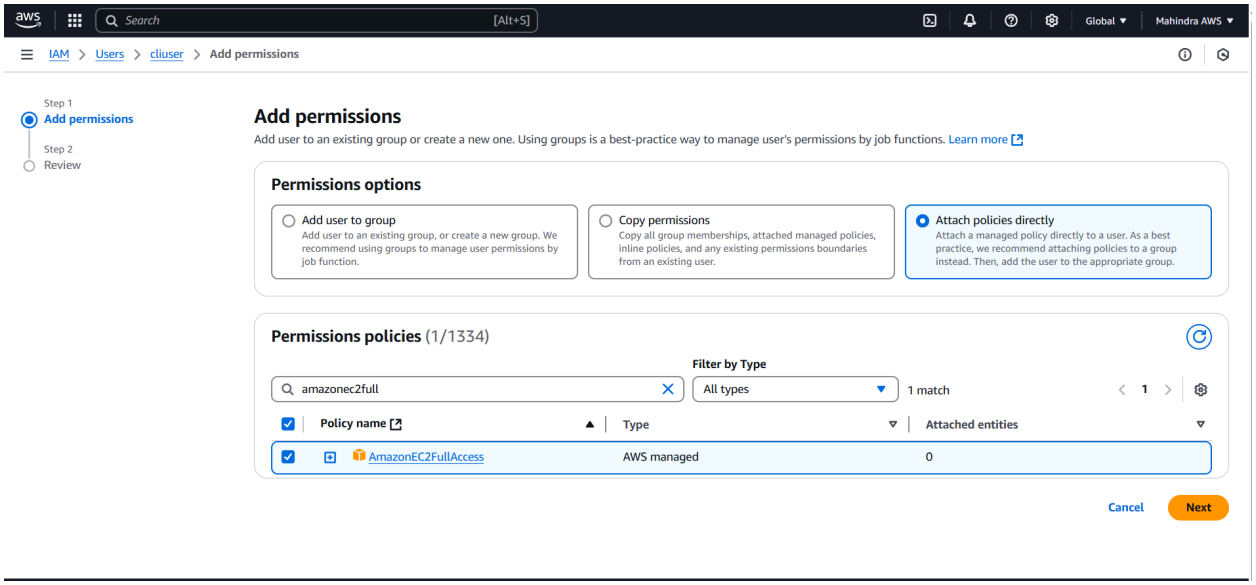
```
 aws ec2 create-subnet --vpc-id vpc-0a3a [redacted]
--cidr-block [redacted] --availability-zone us-east-1b
```



Result:

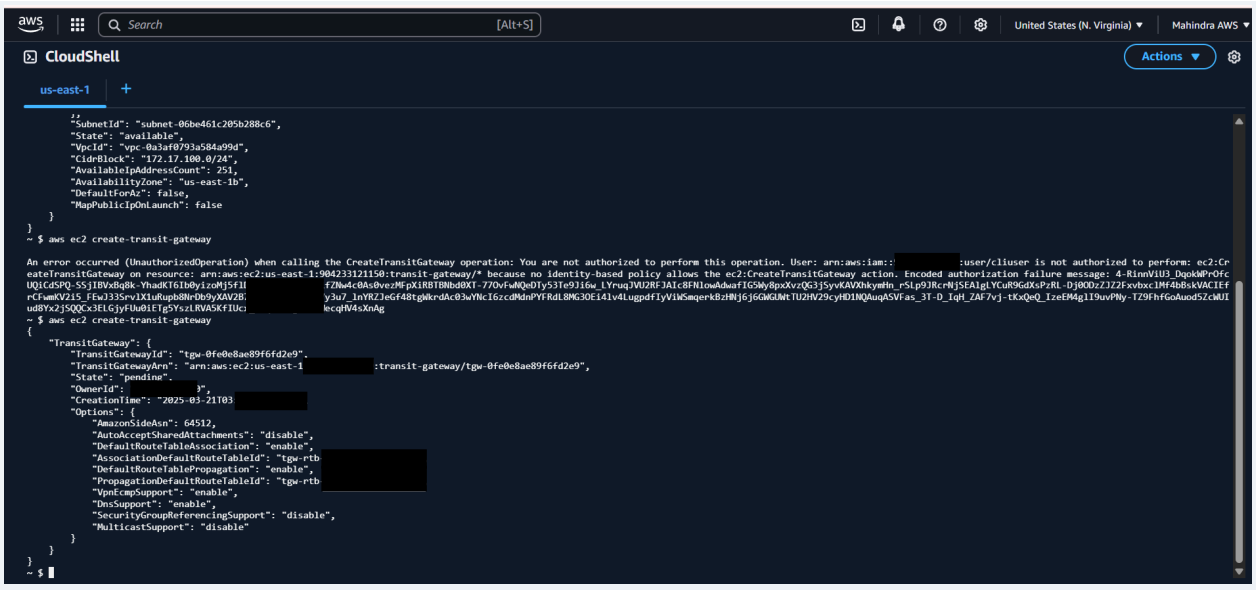

```
{
    "Subnet": {
        "AvailabilityZoneId": "use1-az1",
        "OwnerId": [redacted],
        "AssignIpv6AddressOnCreation": false,
        "Ipv6CidrBlockAssociationSet": [],
        "SubnetArn":
"arn:aws:ec2:us-east-1 [redacted] subnet/subnet-06be461c205b28
8c6",
        "EnableDns64": false,
        "Ipv6Native": false,
        "PrivateDnsNameOptionsOnLaunch": {
            "HostnameType": "ip-name",
            "EnableResourceNameDnsARecord": false,
            "EnableResourceNameDnsAAAARecord": false
        },
        "SubnetId": "subnet-06b [redacted],
        "State": "available",
        "VpcId": "vpc-0a3af [redacted]",
        "CidrBlock": "[redacted],
        "AvailableIpAddressCount": 251,
        "AvailabilityZone": "us-east-1b",
        "DefaultForAz": false,
        "MapPublicIpOnLaunch": false
    }
}
```

Before this you have to add policy to user



Use the following AWS CLI command to create a new transit gateway:

```
aws ec2 create-transit-gateway
```

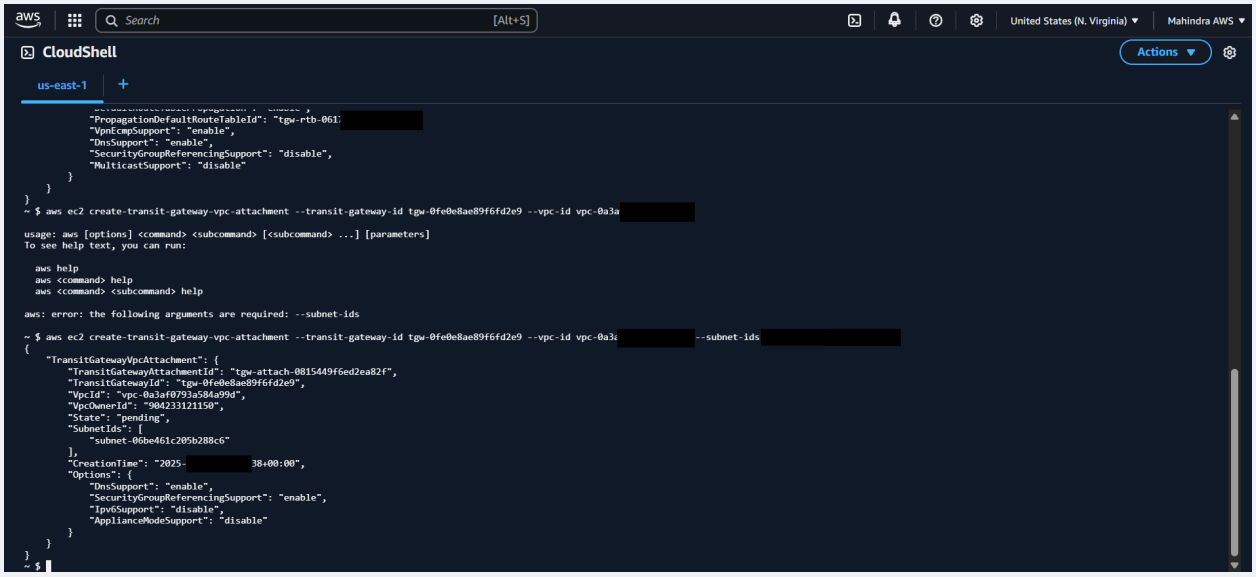


You should see in the output a transit gateway ID and the default transit gateway route table ID:

```
 {
    "TransitGateway": {
        "TransitGatewayId": "tgw-0fe0e8ae89f6fd2e9",
        "TransitGatewayArn":
"arn:aws:ec2:us-east-1 [redacted] transit-gateway/tgw-0fe0e8ae
89f6fd2e9",
        "State": "pending",
        "OwnerId": [redacted] ,
        "CreationTime": "[redacted],
        "Options": {
            "AmazonSideAsn": 64512,
            "AutoAcceptSharedAttachments": "disable",
            "DefaultRouteTableAssociation": "enable",
            "AssociationDefaultRouteTableId":
"tgw-rtb [redacted],
            "DefaultRouteTablePropagation": "enable",
            "PropagationDefaultRouteTableId":
"tgw-rtb [redacted],
            "VpnEcmpSupport": "enable",
            "DnsSupport": "enable",
            "SecurityGroupReferencingSupport": "disable",
            "MulticastSupport": "disable"
        }
    }
}
```

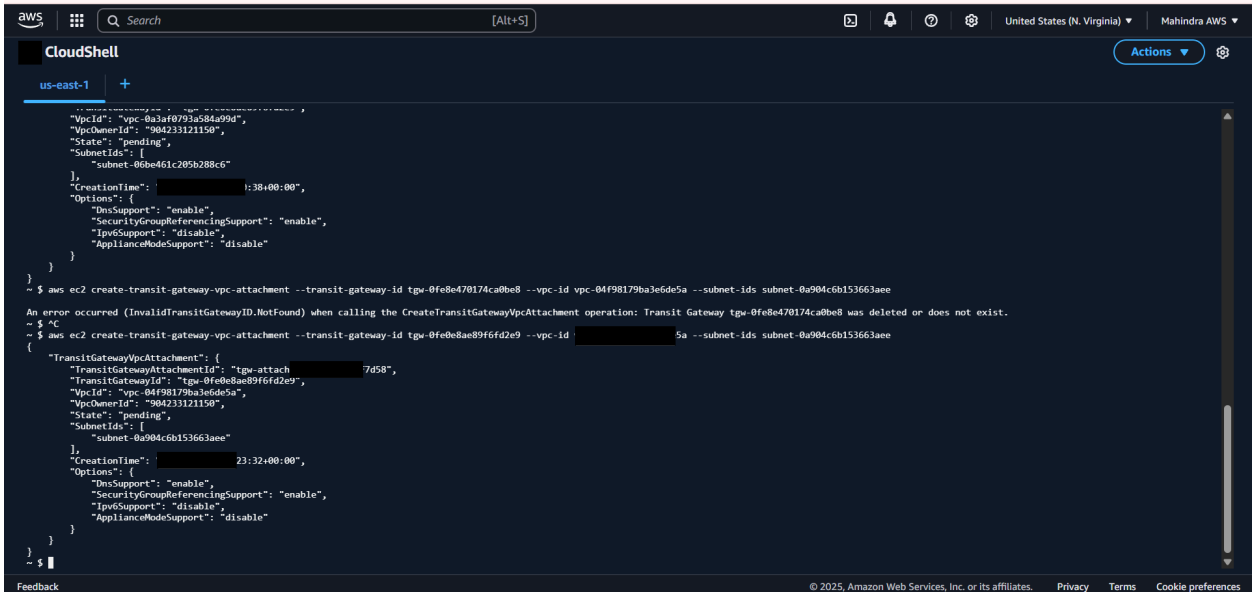
Attach the transit gateway to the VPC subnets:

```
 aws ec2 create-transit-gateway-vpc-attachment
--transit-gateway-id tgw-0fe0e8ae89f6fd2e9 --vpc-id
vpc-0a3a [redacted] --subnet-ids subnet-06b [redacted]
```




```
{
    "TransitGatewayVpcAttachment": {
        "TransitGatewayAttachmentId":
"tgw-attach-0815449f6ed2ea82f",
        "TransitGatewayId": "tgw-0fe0e8ae89f6fd2e9",
        "VpcId": "vpc-0a3a [redacted]",
        "VpcOwnerId": [redacted],
        "State": "pending",
        "SubnetIds": [
            "subnet-06b [redacted]
        ],
        "CreationTime": "[redacted],
        "Options": {
            "DnsSupport": "enable",
            "SecurityGroupReferencingSupport": "enable",
            "Ipv6Support": "disable",
            "ApplianceModeSupport": "disable"
        }
    }
}
```

`aws ec2 create-transit-gateway-vpc-attachment --transit-gateway-id tgw-0fe0e8ae89f6fd2e9 --vpc-id` **vpc-04f98179ba3e6de5a** `--subnet-ids` subnet-[redacted] e

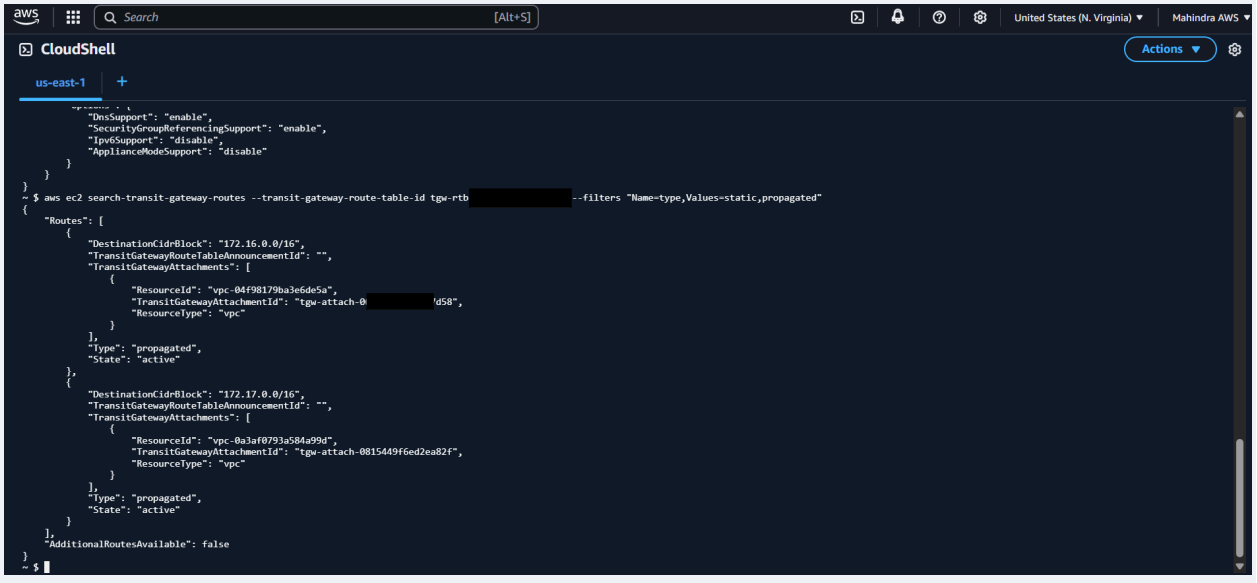



```
{
   "TransitGatewayVpcAttachment": {
      "TransitGatewayAttachmentId": "tgw-attach-[redacted]",
      "TransitGatewayId": "tgw-0fe0e8ae89f6fd2e9",
      "VpcId": "vpc-04f98179ba3e6de5a",
      "VpcOwnerId": "[redacted]",
      "State": "pending",
      "SubnetIds": [
         "subnet-[redacted]ee"
      ],
      "CreationTime": "[redacted]",
      "Options": {
         "DnsSupport": "enable",
         "SecurityGroupReferencingSupport": "enable",
         "Ipv6Support": "disable",
         "ApplianceModeSupport": "disable"
      }
   }
}
```

After you attach the transit gateway to the VPC subnets, the default transit gateway route table is populated with routes to the subnets. You can view these routes with the following command:

```
 aws ec2 search-transit-gateway-routes
--transit-gateway-route-table-id tgw-rtb [redacted] 1
--filters "Name=type,Values=static,propagated"
```

You should see the CIDRs for both subnets as follows:



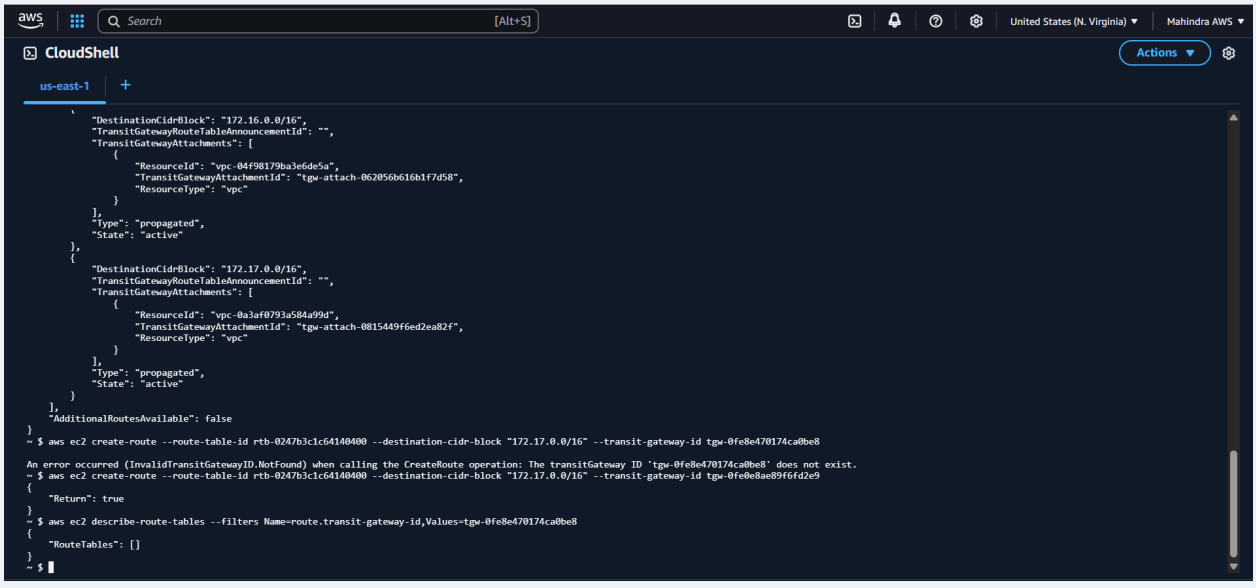

```
{
    "Routes": [
        {
            "DestinationCidrBlock": "[redacted]",
            "TransitGatewayRouteTableAnnouncementId": "",
            "TransitGatewayAttachments": [
                {
                    "ResourceId": "vpc-04f98179ba3e6de5a",
                    "TransitGatewayAttachmentId":
"tgw-attach [redacted]",
                    "ResourceType": "vpc"
                }
            ],
            "Type": "propagated",
            "State": "active"
        },
        {
            "DestinationCidrBlock": "[redacted]",
            "TransitGatewayRouteTableAnnouncementId": "",
```

```
            "TransitGatewayAttachments": [
                {
                    "ResourceId": "vpc-0a3a [redacted]",
                    "TransitGatewayAttachmentId":
"tgw-attach-0815449f6ed2ea82f",
                    "ResourceType": "vpc"
                }
            ],
            "Type": "propagated",
            "State": "active"
        }
    ],
    "AdditionalRoutesAvailable": false
}
```

The transit gateway is now configured to pass traffic between the subnets.
To use the transit gateway, you must add to each subnet's route table a route for the other subnet.
To do this, you'll need the route table IDs and CIDRs for each subnet.

```
 aws ec2 create-route --route-table-id rtb- [redacted]
--destination-cidr-block "[redacted] --transit-gateway-id
tgw-0fe0e8ae89f6fd2e9
```

```
 aws ec2 create-route
 --route-table-id rtb-01bbf[redacted]
 --destination-cidr-block "[redacted] -transit-gateway-id
tgw-0fe8e470174ca0be8
```



Verify the routes using the following command:

```
 aws ec2 describe-route-tables --filters
Name=route.transit-gateway-id,Values=tgw-0fe8e470174ca0be8
```

Your output should look like this:

```
 {
    "RouteTables": [
        {
            "Associations": [
                {
                    "Main": true,
                    "RouteTableAssociationId":
"rtbassoc-00f60edb255be0332",
                    "RouteTableId": "rtb-097d8b [redacted],
                    "AssociationState": {
                        "State": "associated"
                    }
                }
            ],
            "PropagatingVgws": [],
            "RouteTableId": "rtb-097d8b [redacted],
            "Routes": [
                {
                    "DestinationCidrBlock": "[redacted],
                    "GatewayId": "local",
                    "Origin": "CreateRouteTable",
                    "State": "active"
                },
                {
                    "DestinationCidrBlock": "[redacted],
                    "TransitGatewayId": "tgw-0fe8e470174ca0be8",
                    "Origin": "CreateRoute",
                    "State": "active"
                },
                {
                    "DestinationCidrBlock": "0.0.0.0/0",
                    "GatewayId": "igw-0312f81aa1ef24715",
                    "Origin": "CreateRoute",
                    "State": "active"
                }
            ],
            "Tags": [],
```

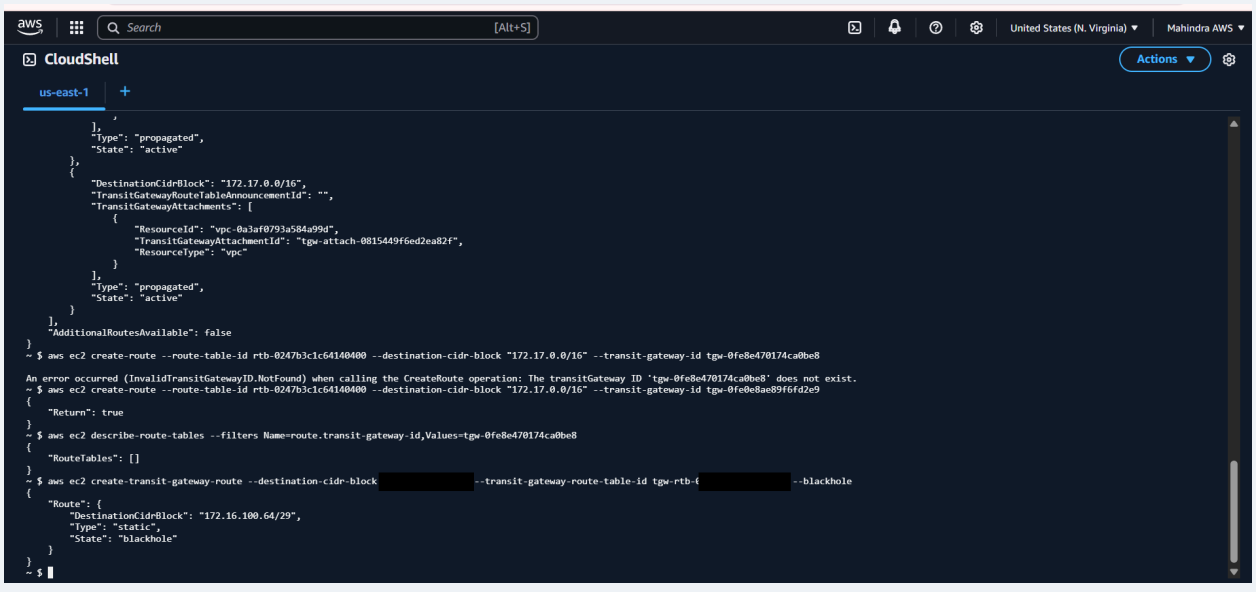
```
            "VpcId": "vpc [redacted]d",
            "OwnerId": [redacted]
        },
        {
            "Associations": [
                {
                    "Main": true,
                    "RouteTableAssociationId":
"rtbassoc-051c79bc6d208c008",
                    "RouteTableId": "rtb-01bb [redacted],
                    "AssociationState": {
                        "State": "associated"
                    }
                }
            ],
            "PropagatingVgws": [],
            "RouteTableId": "rtb-01bb [redacted],
            "Routes": [
                {
                    "DestinationCidrBlock": "[redacted],
                    "TransitGatewayId": "tgw-0fe8e470174ca0be8",
                    "Origin": "CreateRoute",
                    "State": "active"
                },
                {
                    "DestinationCidrBlock": "[redacted],
                    "GatewayId": "local",
                    "Origin": "CreateRouteTable",
                    "State": "active"
                }
            ],
            "Tags": [],
            "VpcId": "vpc-08edadb7e52eedd37",
            "OwnerId": [redacted]
        }
    ]
```

EXERCISE 4.9

Create a Blackhole Route

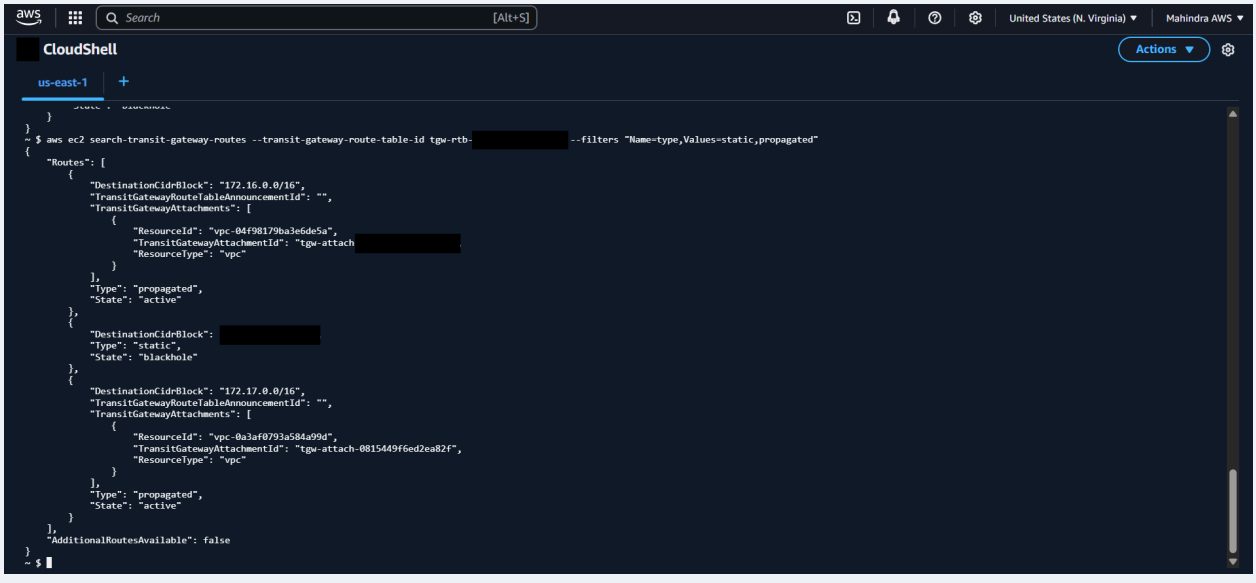
In this exercise, you'll create a blackhole route for the range of addresses [redacted] 172.16.100.71. You'll then delete your transit gateway. Use the following command to create a blackhole route for the [redacted] subnet:

```
aws ec2 create-transit-gateway-route --destination-cidr-block [redacted] --transit-gateway-route-table-id tgw-rtb [redacted] 1 --blackhole
```



When you're done practicing, you'll want to delete the transit gateway attachments to avoid charges. You'll need the transit gateway attachment IDs, which you can get using the same command you used in Exercise 4.8:

```
aws ec2 search-transit-gateway-routes --transit-gateway-route-table-id tgw-rtb [redacted] 1 --filters "Name=type,Values=static,propagated"
```



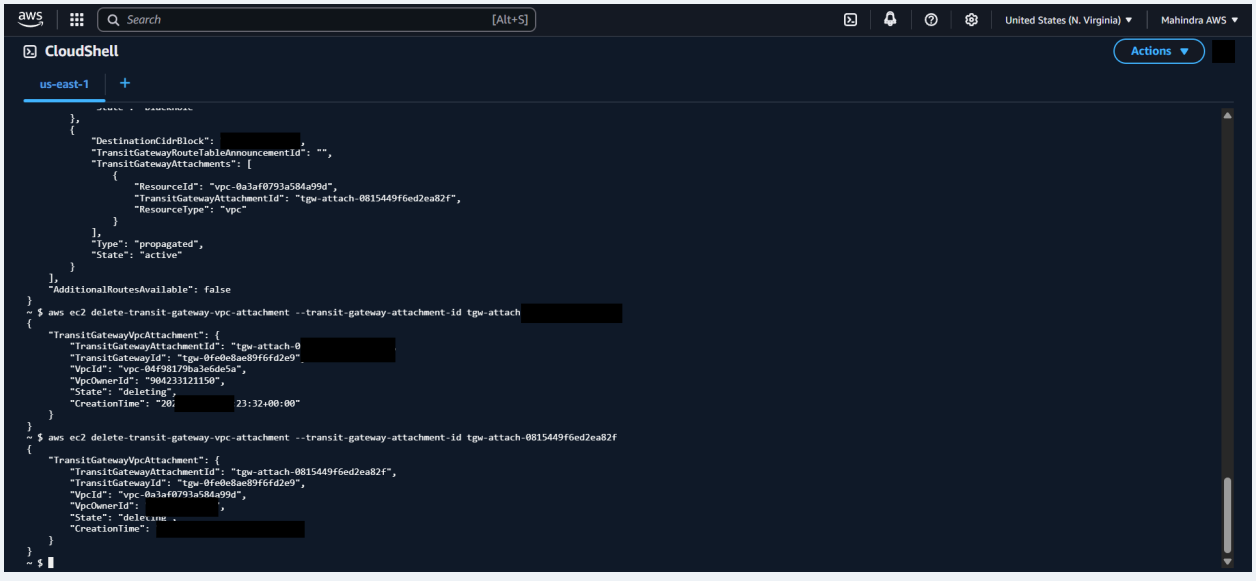

```
{
    "Routes": [
        {
            "DestinationCidrBlock": "[redacted],
            "TransitGatewayRouteTableAnnouncementId": "",
            "TransitGatewayAttachments": [
                {
                    "ResourceId": "vpc-04f98179ba3e6de5a",
                    "TransitGatewayAttachmentId":
"tgw-attach [redacted],
                    "ResourceType": "vpc"
                }
            ],
            "Type": "propagated",
            "State": "active"
        },
        {
            "DestinationCidrBlock": "[redacted],
            "Type": "static",
            "State": "blackhole"
        },
        {
            "DestinationCidrBlock": "[redacted],
            "TransitGatewayRouteTableAnnouncementId": "",
            "TransitGatewayAttachments": [
                {
                    "ResourceId": "vpc-0a3af[redacted]",
```

```
                    "TransitGatewayAttachmentId":
"tgw-attach-0815449f6ed2ea82f",
                    "ResourceType": "vpc"
                }
            ],
            "Type": "propagated",
            "State": "active"
        }
    ],
    "AdditionalRoutesAvailable": false
}
```

Delete the attachments using the following commands:

```
 aws ec2 delete-transit-gateway-vpc-attachment
--transit-gateway-attachment-id tgw-attach [redacted]
```

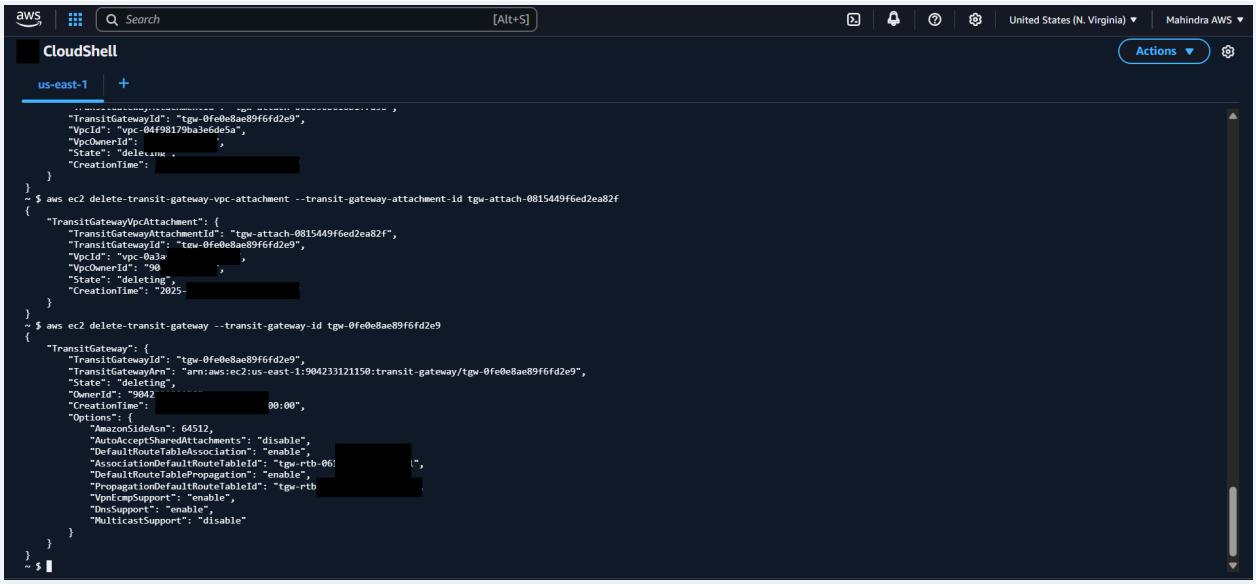
```
 aws ec2 delete-transit-gateway-vpc-attachment
--transit-gateway-attachment-id tgw-attach-0815449f6ed2ea82f
```



It may take a few minutes for the attachments to be deleted.

Delete the transit gateway using the following command:

`aws ec2 delete-transit-gateway --transit-gateway-id tgw-0fe0e8ae89f6fd2e9`

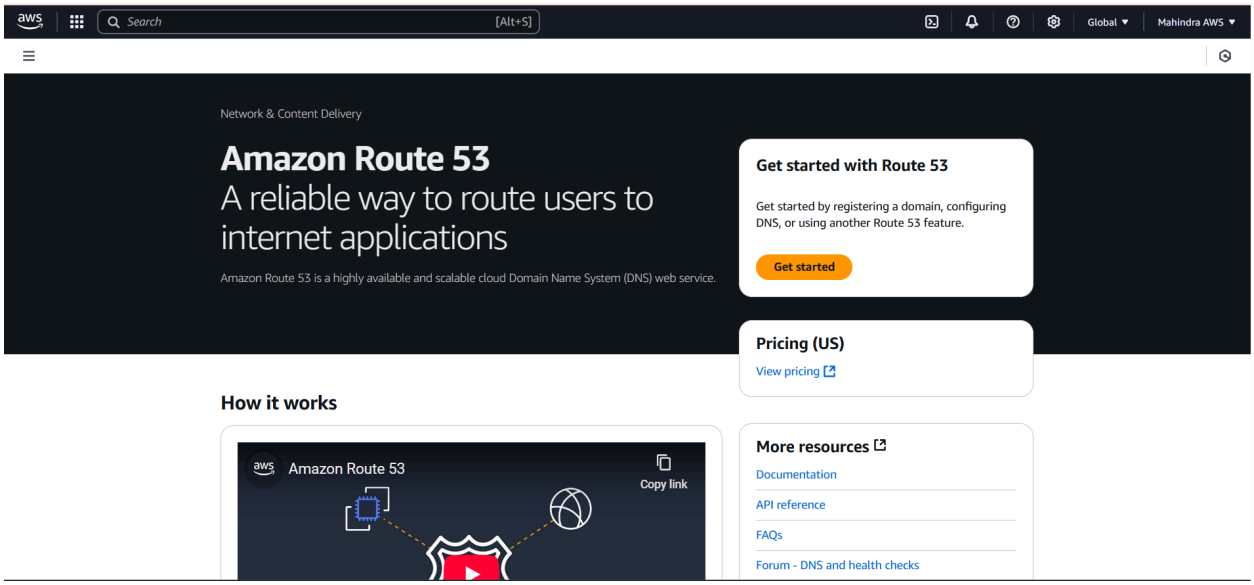


Chapter 8

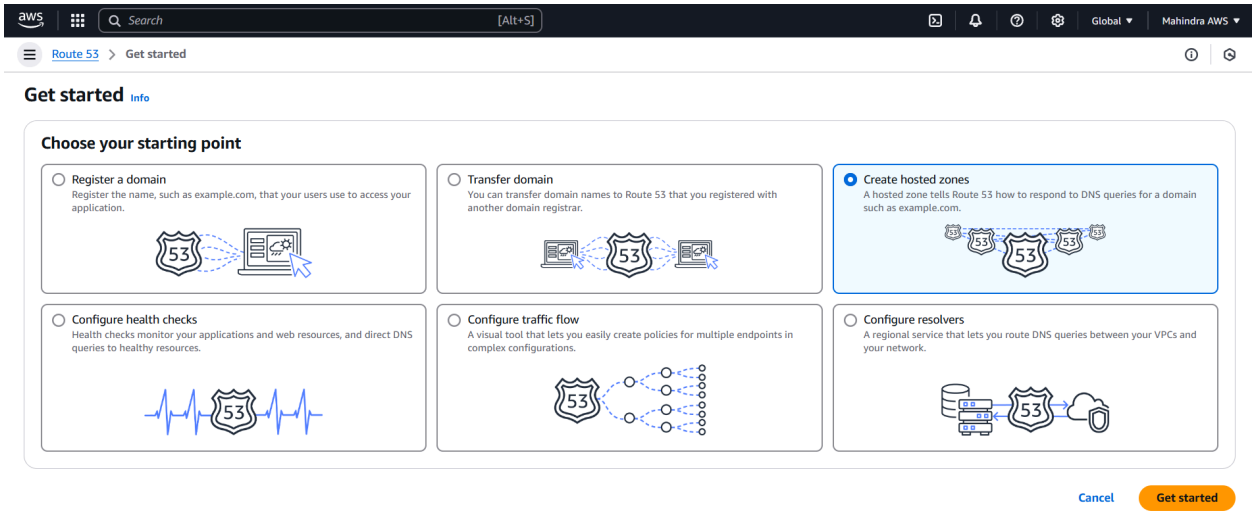
EXERCISE 8.1

Create a Hosted Zone on Route 53 for an EC2 Web Server

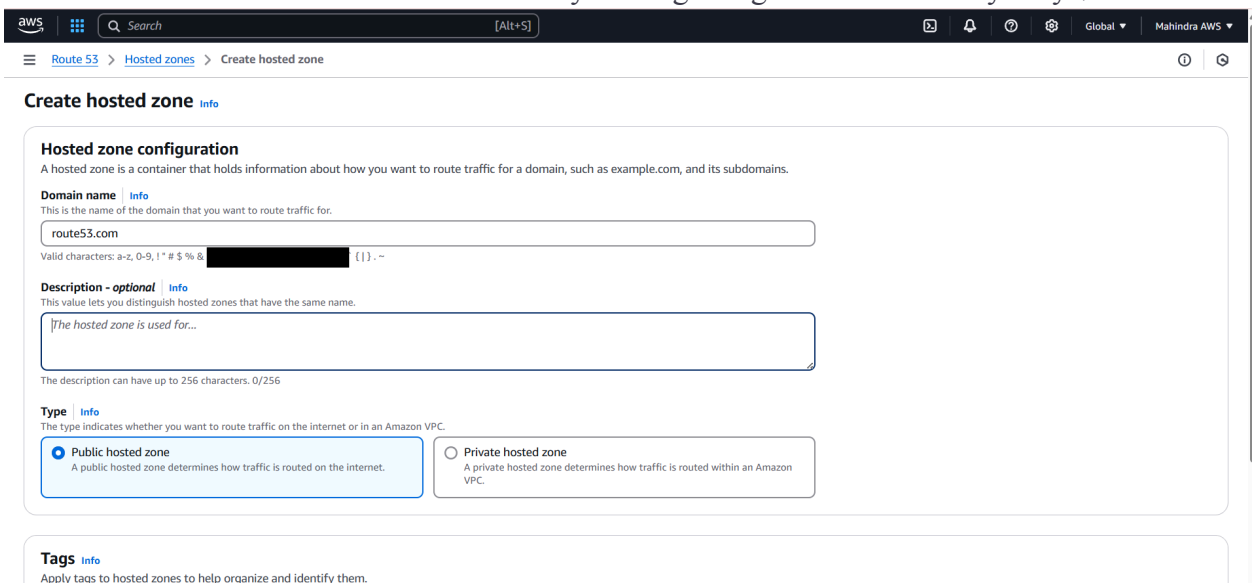
1. Click the Hosted Zones link and then the Create Hosted Zone button in the Route 53 dashboard.

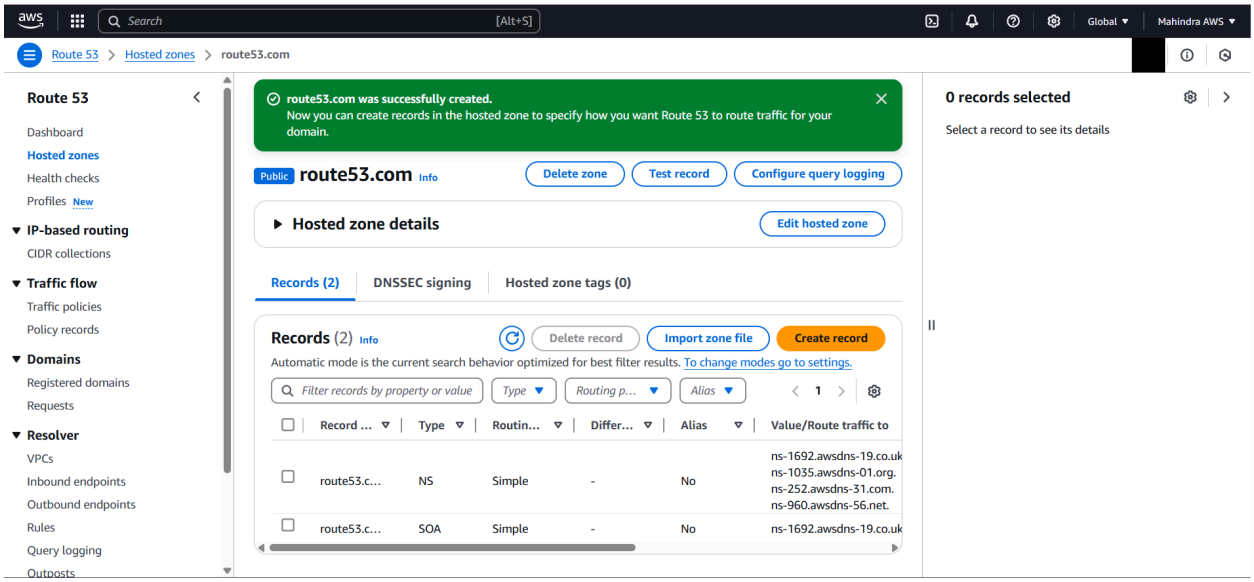


Click on get started and select create hosted zone.

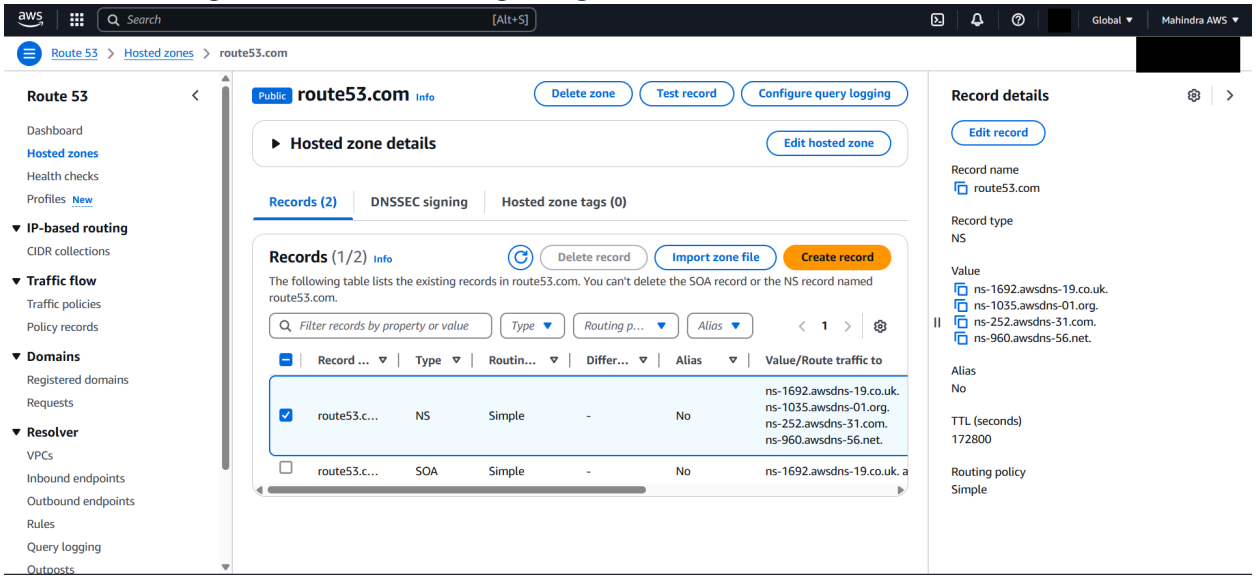


2. Enter a valid domain name that's currently not managed and select the Public Hosted Zone type. If you don't already have an existing domain on Route 53 and don't want to create a new one just for this exercise, use a fake domain—even something like example.com. Of course, you won't be able to fully test the zone if you're not using a real domain name. Either way, remember that leaving the hosted zone up for more than 12 hours will incur a monthly hosting charge—that's currently only $0.50.

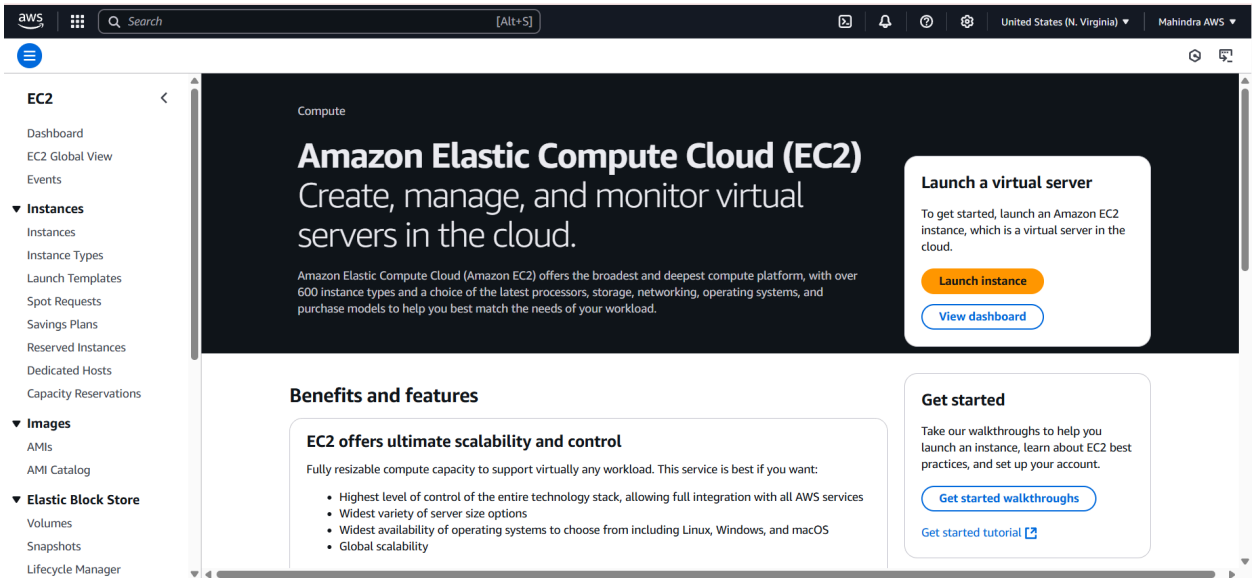




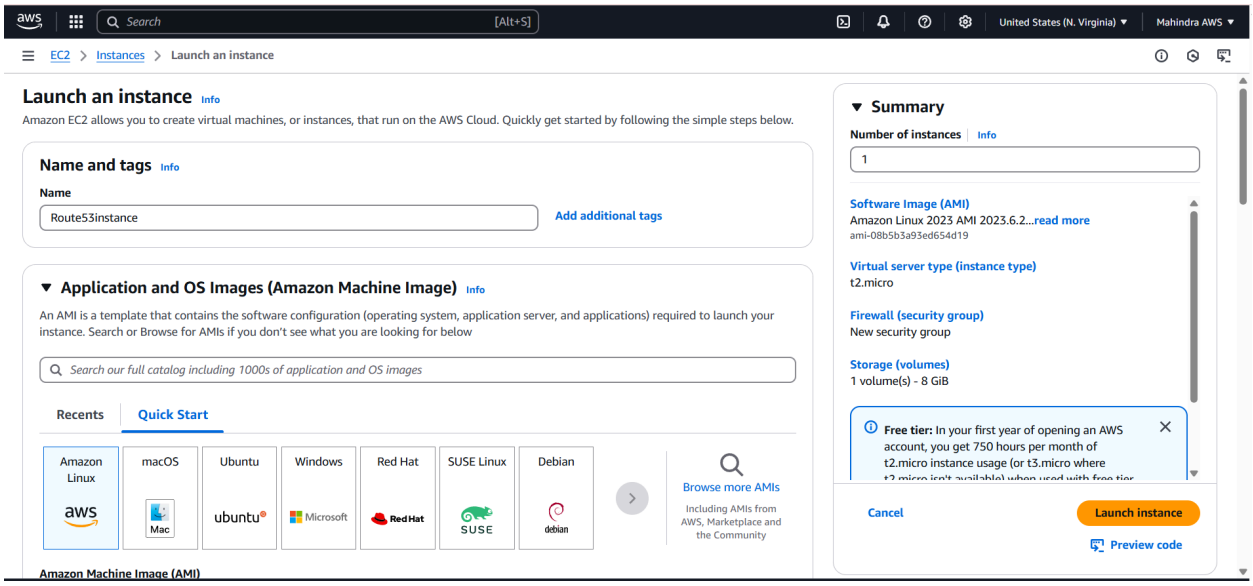
3. You'll be given name server and SOA record sets. Select those records one at a time and spend a few minutes exploring.

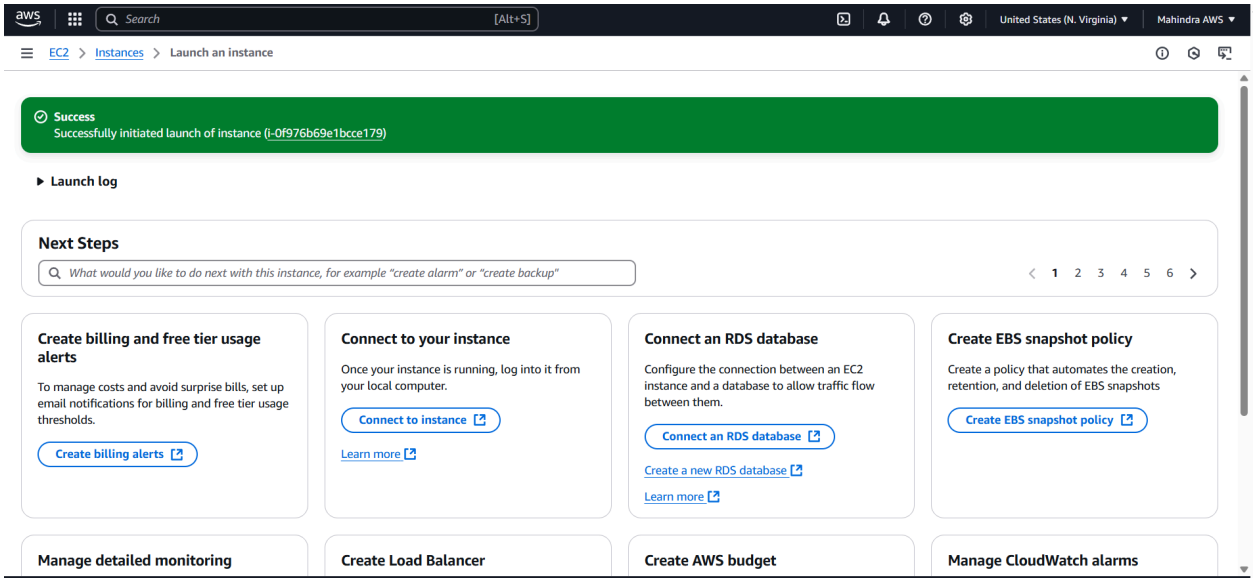


4. From the Amazon EC2 Dashboard,



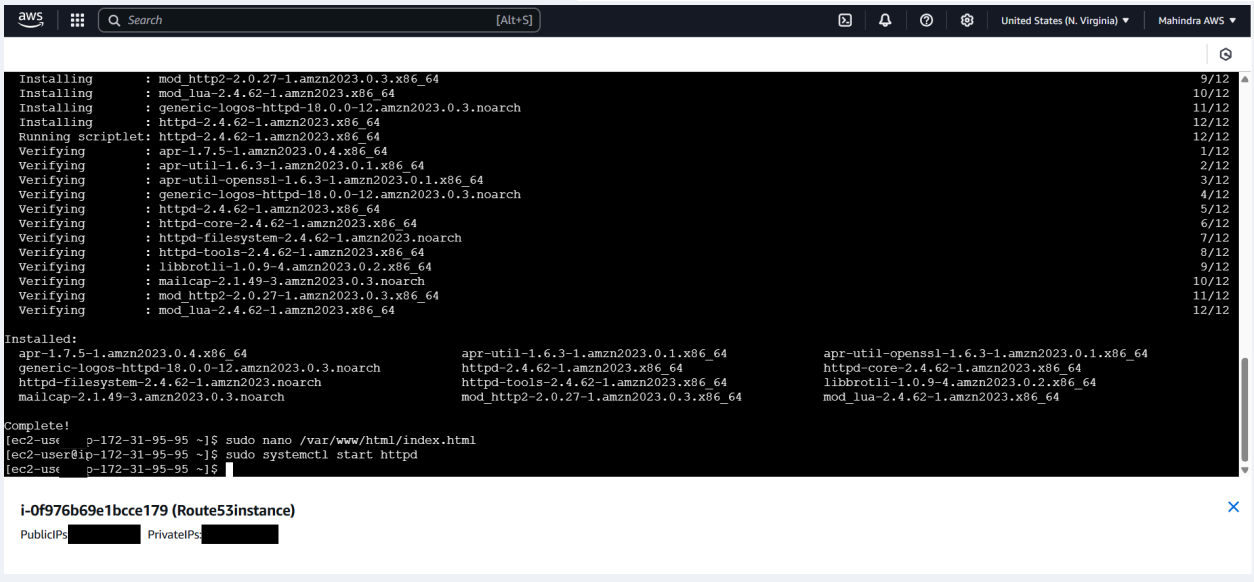
create an AWS Linux AMI-based instance the way you did in Exercise 2.1 in Chapter 2, "Compute Services." You'll configure this instance as a simple web server so that you can test the domain. When configuring your security group, confirm that port 22 (SSH) is open and open HTTP port 80 (and HTTPS port 443 if necessary) so that web traffic can get in.



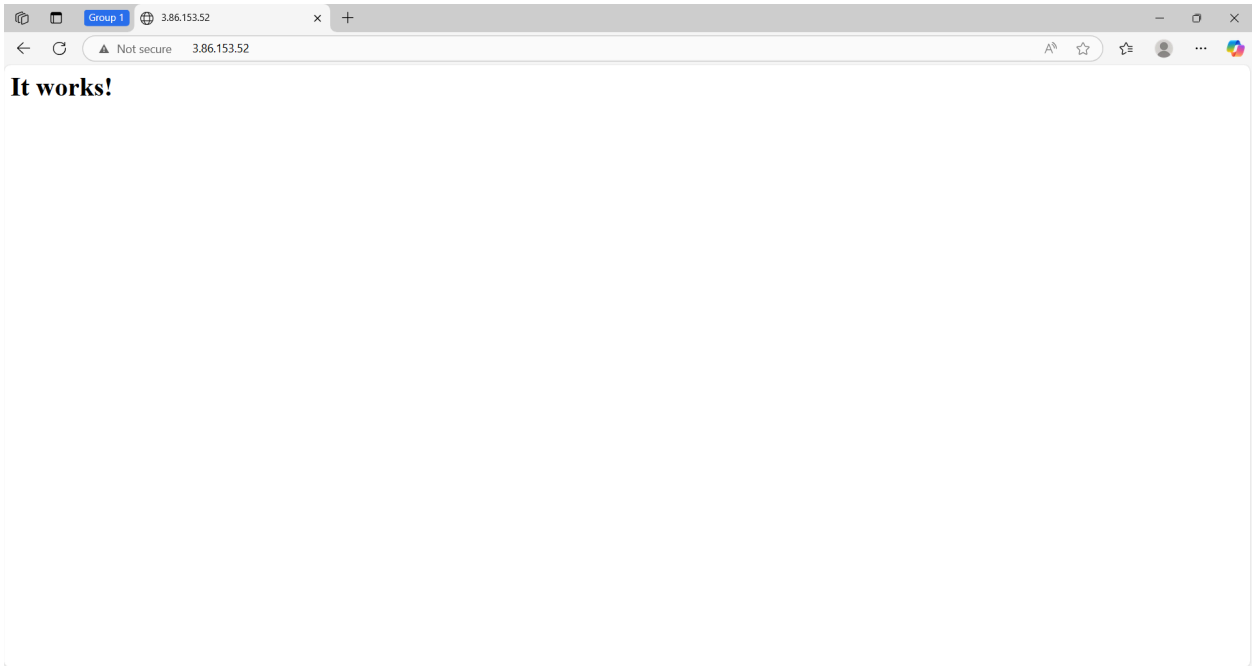


Log into your instance (as you did in Exercise 2.1) and run the following commands to update the software repository, install the Apache web server, create a default `index.html` web page (add some text to the page when the text editor opens, and, when you're done, save and exit using Ctrl+X and then Y; this will later help you know that everything is working), and start up the Apache process.

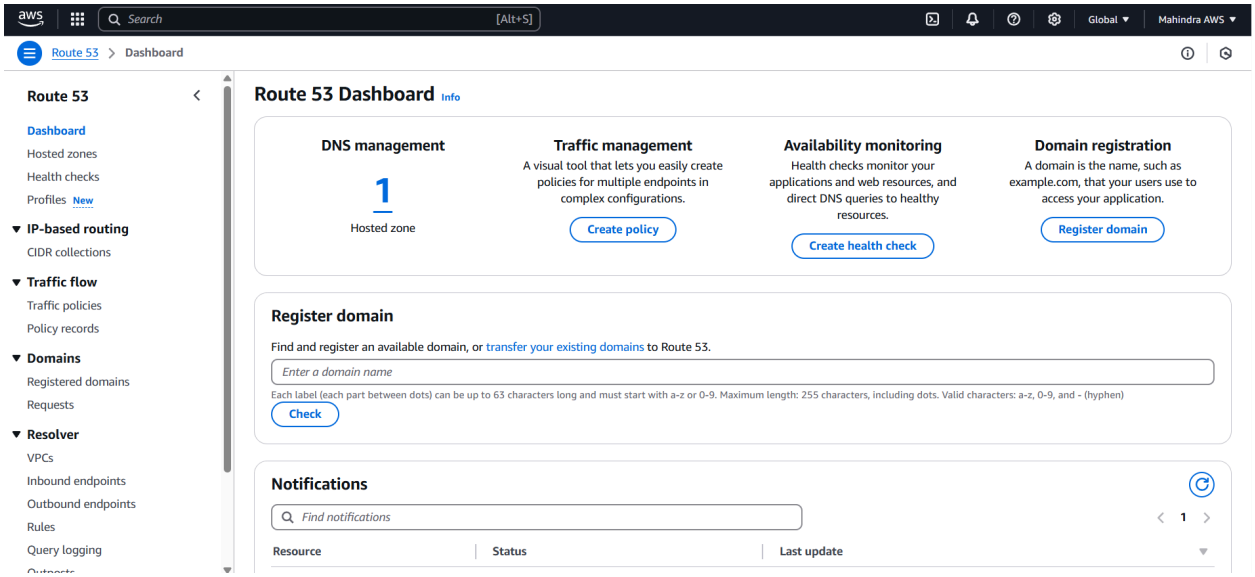
```
sudo yum update -y
sudo yum install -y httpd
sudo nano /var/www/html/index.html
sudo systemctl start httpd
```

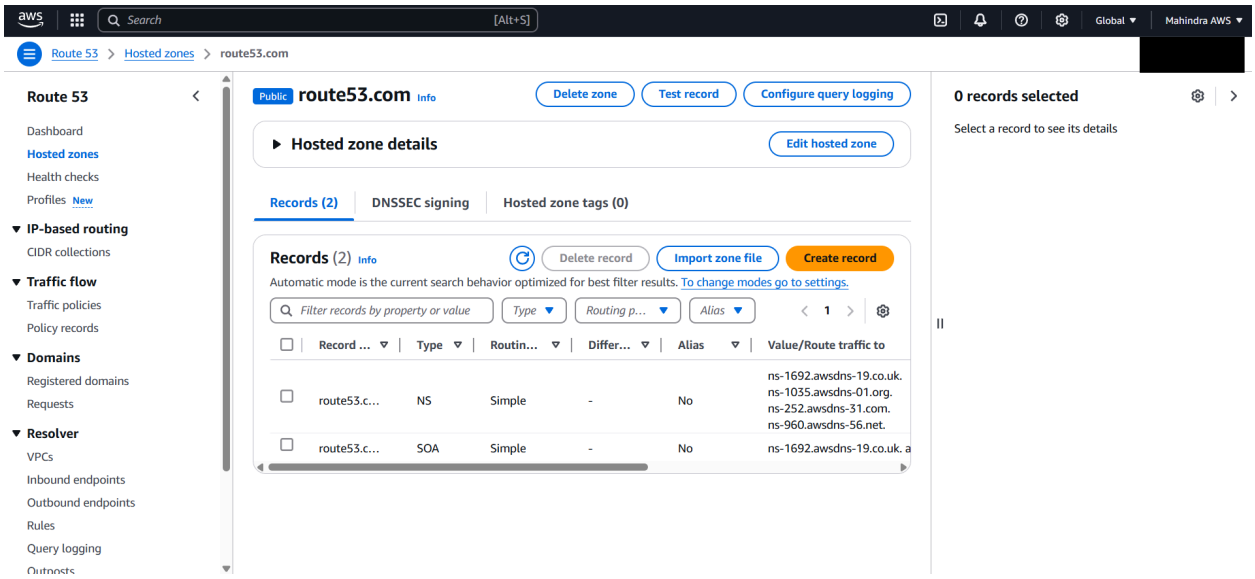


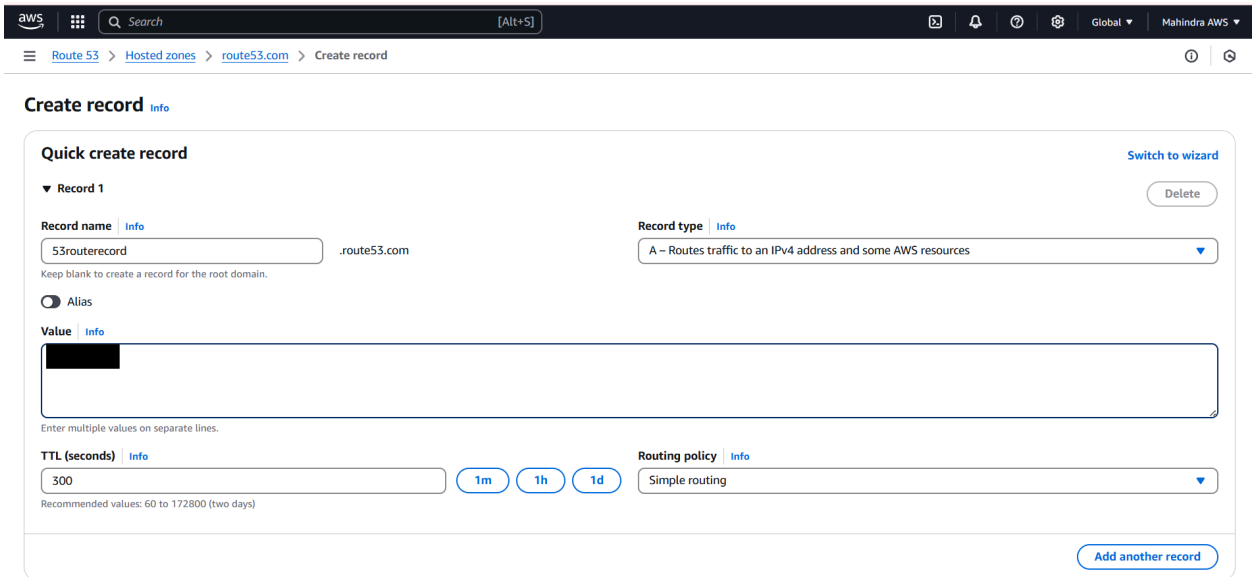
5. You can confirm your web server is working by pointing your browser to the instance's public IP address. You may want to keep this instance running, as it will also be helpful for the next exercise.

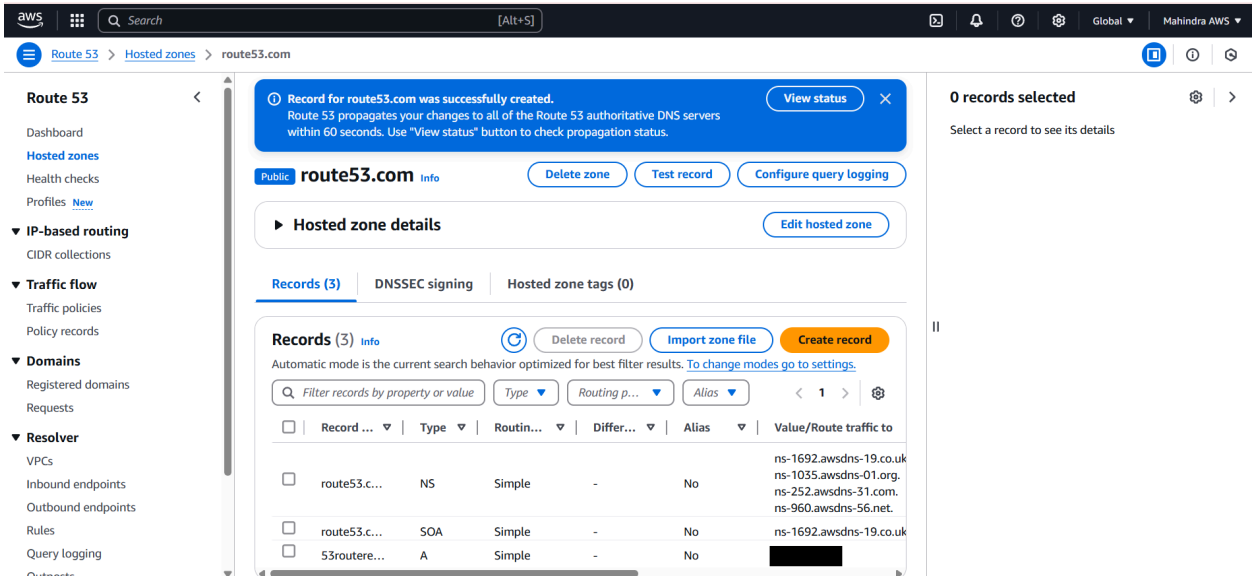


6. Back in the Route 53 Dashboard, click the Create Record button to create an A record to map your domain name to the IP address of a web server instance running on EC2. Leave the Name field to the left of the example.com value blank and enter your server's IP address (or any realistic fake address if your domain isn't real) in the Value field. Click Create.

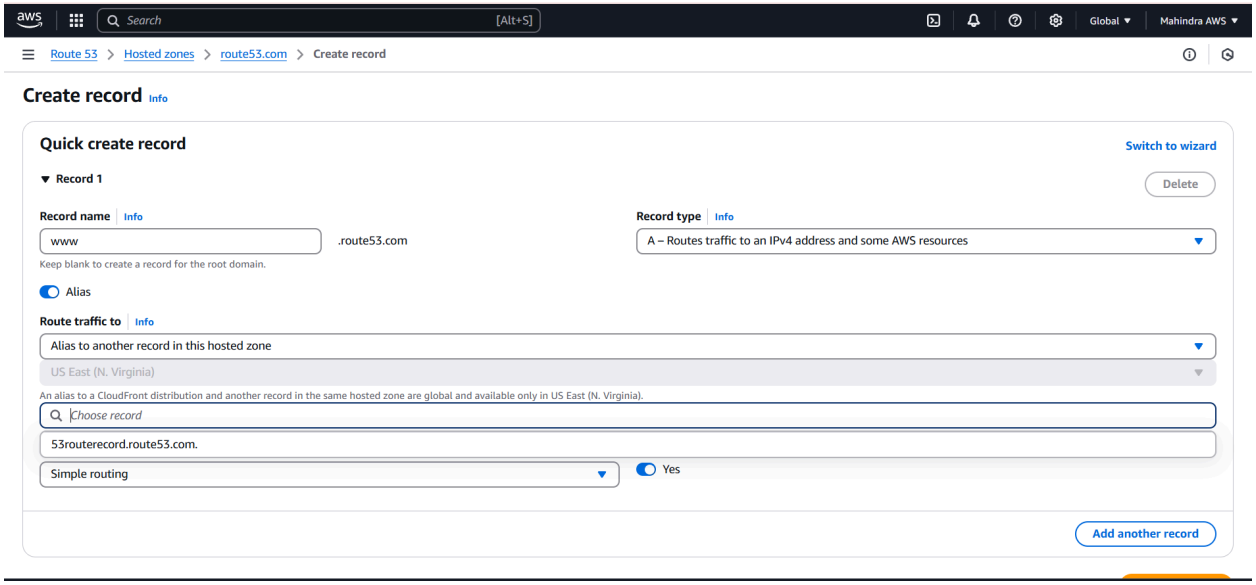



Search [Alt+S]
Global
Mahindra AWS
Route 53 > Hosted zones > route53.com
Route 53
Dashboard
Hosted zones
Health checks
Profiles New
IP-based routing
CIDR collections
Traffic flow
Traffic policies
Policy records
Domains
Registered domains
Requests
Resolver
VPCs
Inbound endpoints
Outbound endpoints
Rules
Query logging
Outposts
Public route53.com Info
Delete zone
Test record
Configure query logging
Hosted zone details
Edit hosted zone
Records (2)
DNSSEC signing
Hosted zone tags (0)
Records (2) Info
Delete record
Import zone file
Create record
Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.
Filter records by property or value
Type
Routing p...
Alias
1
Record ...
Type
Routin...
Differ...
Alias
Value/Route traffic to
route53.c...
NS
Simple
-
No
ns-1692.awsdns-19.co.uk.
ns-1035.awsdns-01.org.
ns-252.awsdns-31.com.
ns-960.awsdns-56.net.
route53.c...
SOA
Simple
-
No
ns-1692.awsdns-19.co.uk. a
0 records selected
Select a record to see its details

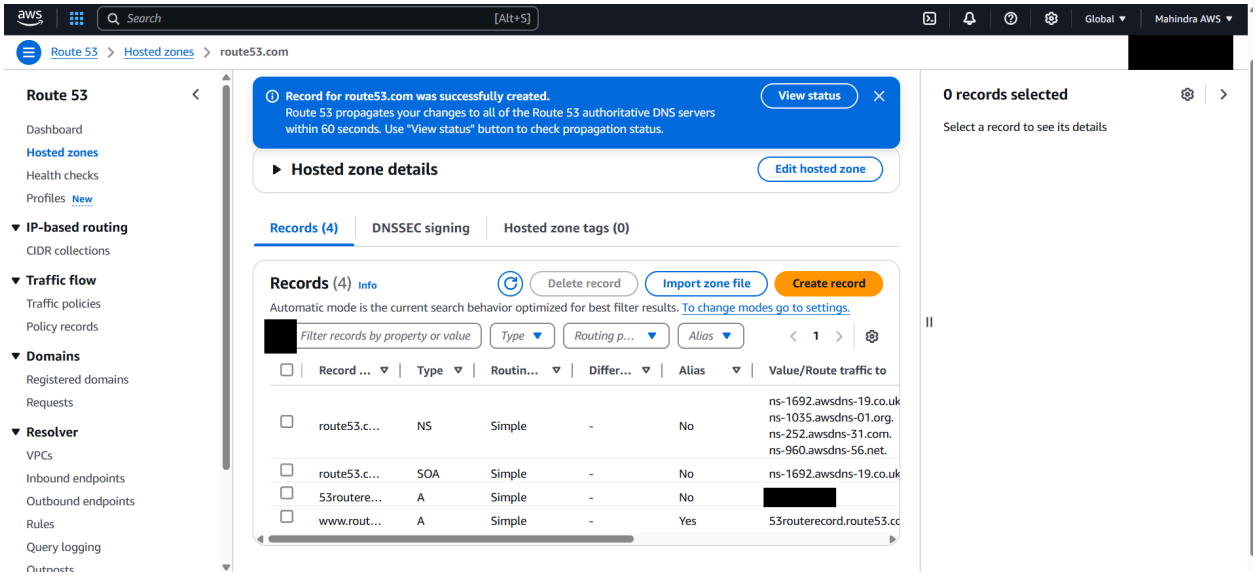

Search [Alt+S]
Global
Mahindra AWS
Route 53 > Hosted zones > route53.com > Create record
Create record Info
Quick create record
Switch to wizard
Record 1
Delete
Record name Info
53routerecord
.route53.com
Keep blank to create a record for the root domain.
Record type Info
A – Routes traffic to an IPv4 address and some AWS resources
Alias
Value Info
Enter multiple values on separate lines.
TTL (seconds) Info
300
1m
1h
1d
Recommended values: 60 to 172800 (two days)
Routing policy Info
Simple routing
Add another record



7. Click Create Record Set once again to create a second A record. This time, you'll enter **www** in the Name field.
8. Click the Yes radio button next to Alias, click once inside the Alias Target field, and select the example.com value that should be among those that appear.



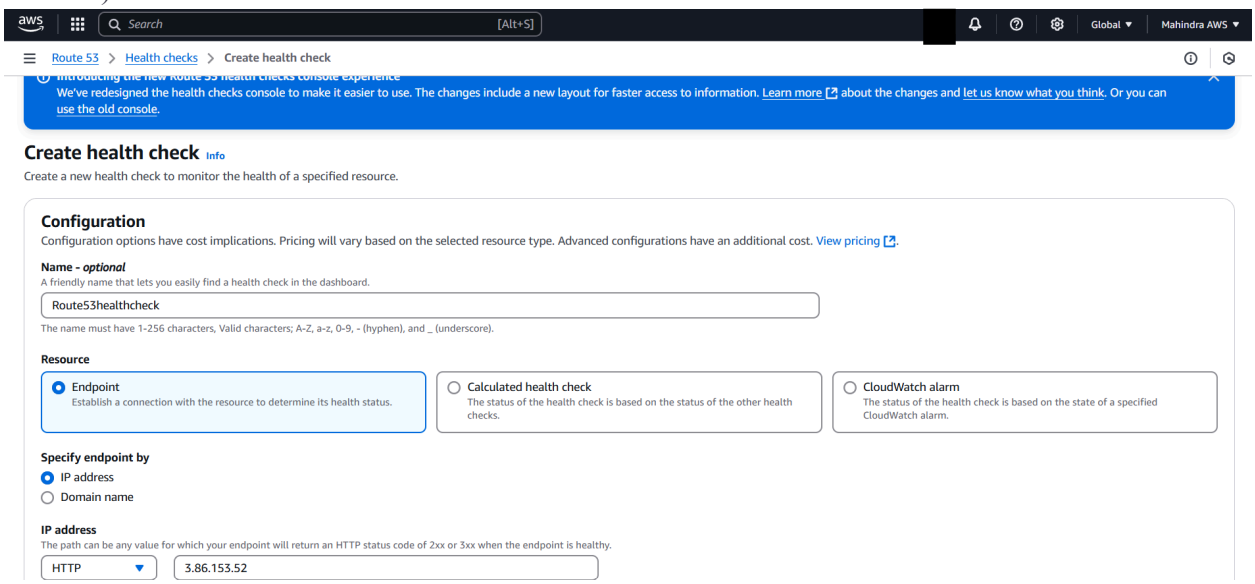
9. Domain propagation might take a few hours, but assuming you're working with an actual server, you should eventually be able to point your browser to example.com and your server's root web page should load.



EXERCISE 8.2

Set Up a Health Check

1. In the Route 53 GUI, click Create Health Check.
2. Give your check a name and select the Endpoint radio button for What To Monitor. You can use the web server you created in Exercise 8.1 as the health check endpoint.
3. With the IP address radio button selected for Specify Endpoint By, choose either HTTP or HTTPS for the protocol (this will depend on how you configured Transport Layer Security [TLS] encryption for your website—HTTP is a good bet if you're not sure).



4. Enter your server's IP address, and enter either **80** or **443** for the Port value (again, this will depend on your site's TLS settings—80 is a good bet if you're not sure).
5. Enter the name of a file in the web root directory of your website as the Path value. This is the resource the health check will try to load to test your site's health. You

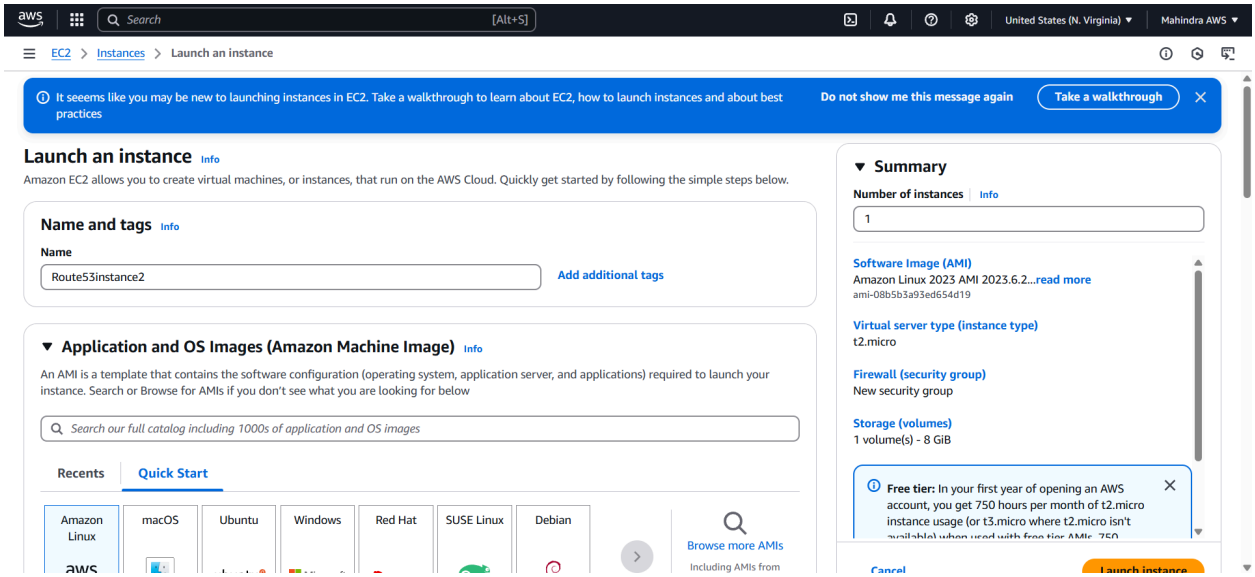
could use `index.html` (or `index.php`, if you happen to have a PHP application running), but, over the long term, that will probably cause a lot of unnecessary overhead. Instead, consider creating a small file called something like `test.html` and entering that for Path.

6. You can create an alarm to alert you to failed tests or simply click Create Health Check

EXERCISE 8.3

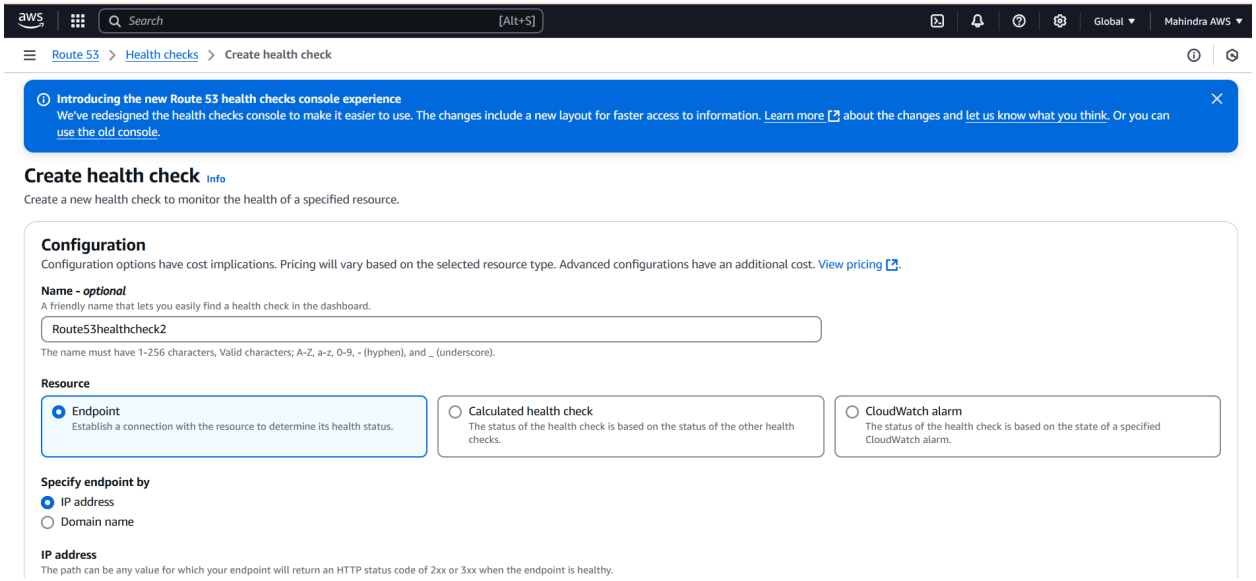
Configure a Route 53 Routing Policy

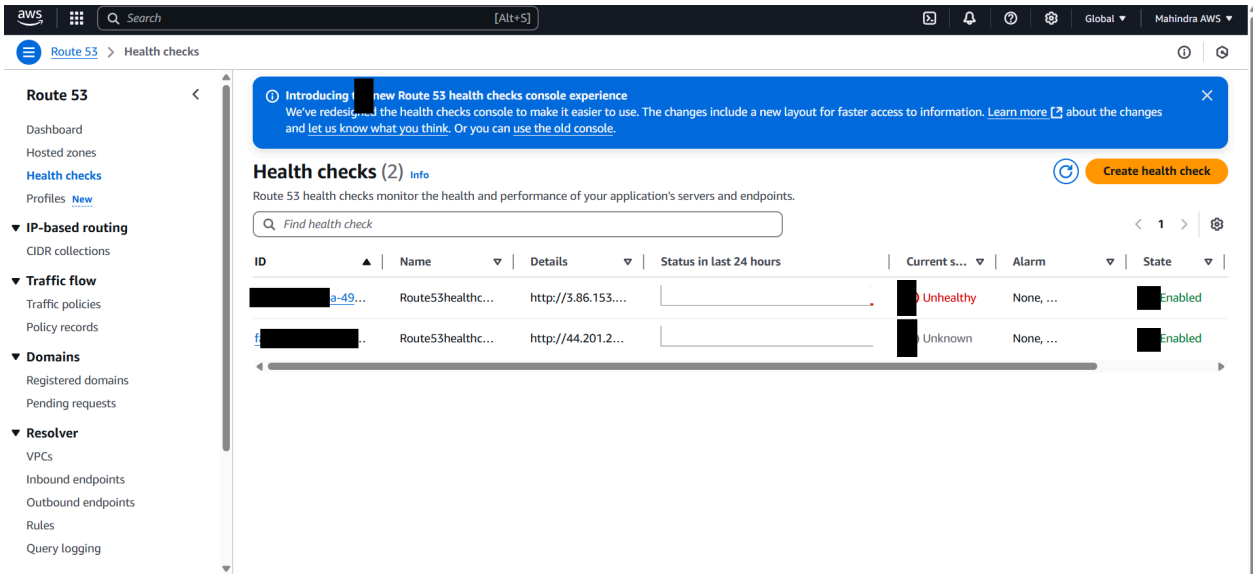
1. Make sure you have two separate web-facing resources running. You could create a second Apache web server instance in addition to the one you launched in Exercise 8.1, or perhaps create a simple static website in S3 the way you did in Exercise 3.4 in Chapter 3, "AWS Storage." In this case, though, your bucket name will have to exactly match the domain name. That might mean something like secondary.example.com.



2. Configure a health check for each of your resources. You create a health check for an S3 static website by selecting Domain Name for Specify Endpoint By. Note that you'll need to strip off the `http://` and trailing `/` characters from the endpoint before the health check configuration will accept it.
3. Assuming you have an active hosting zone (perhaps the one from Exercise 8.1), use the Route 53 GUI to create a record set for each of your instances. You might enter domain names like server1.example.com and server2.example.com—it's up to you.
4. Create a regular A record type if you're pointing to an EC2 instance, and enter the instance's IP address as the record's Value. If you're using an S3 static website for one or both of your resources, click Yes for Alias in the record set, click once within the Alias Target box that will appear, and select the appropriate S3 resource from among those that appear. Your static website must live in the same AWS account.
5. For Routing Policy, select Failover and choose either Primary or Secondary from the Failover Record Type drop-down menu. The Set ID value will automatically change according to your choice. Make sure the record set for one of your resources is designated as Primary.

6. Select Yes for Evaluate Target Health, and select Yes for Associate With Health Check. Select the health check that matches the resource you're working with.
7. Test your configuration by first loading the primary website by its normal URL. Now, disable your primary website. You could do that by deleting or renaming its `index.html` file, by blocking HTTP access in the instance security group, or in the case of an S3, by disabling the static website setting. Point your browser to the same primary URL. If failover is working, you should see the `index.html` web page from your secondary resource.

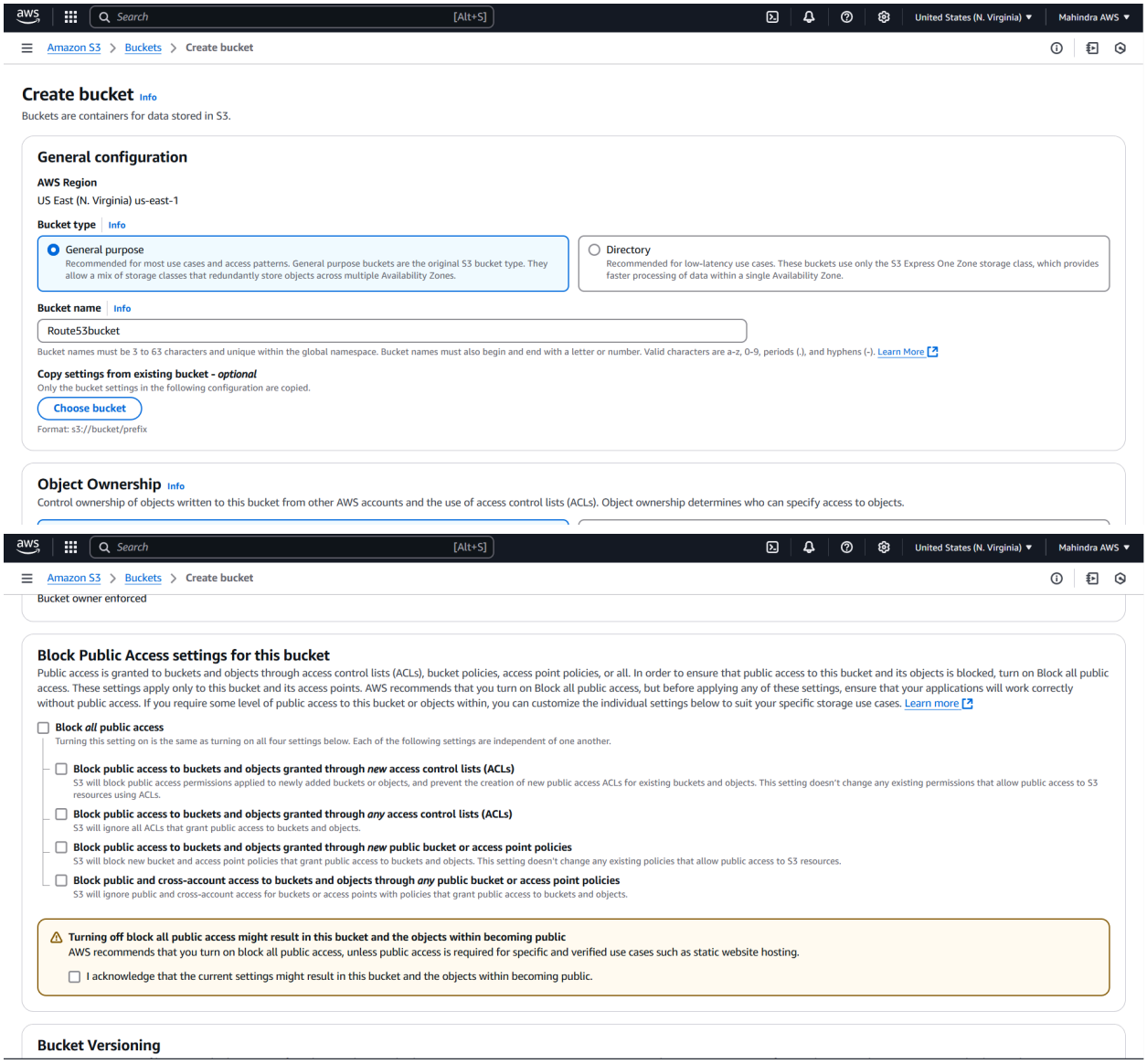


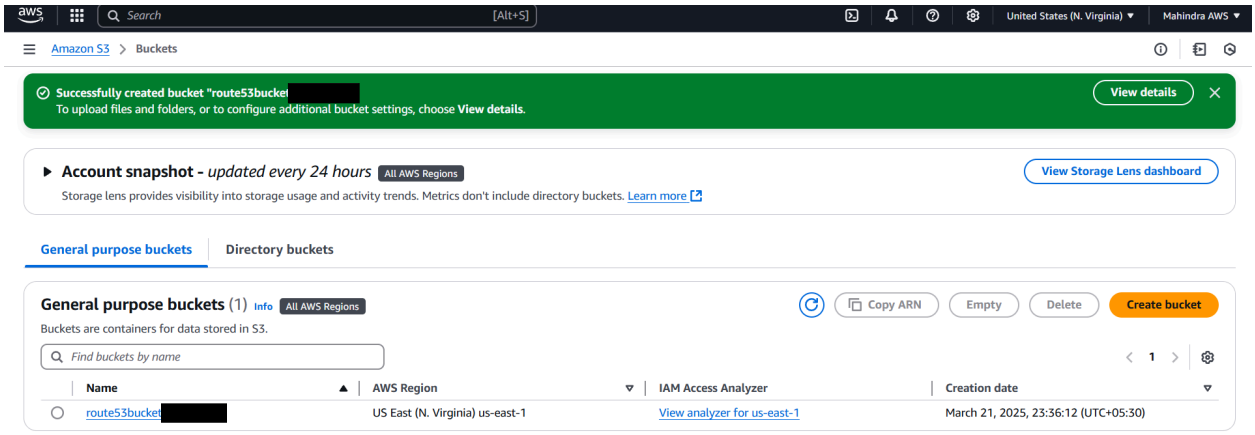


EXERCISE 8.4

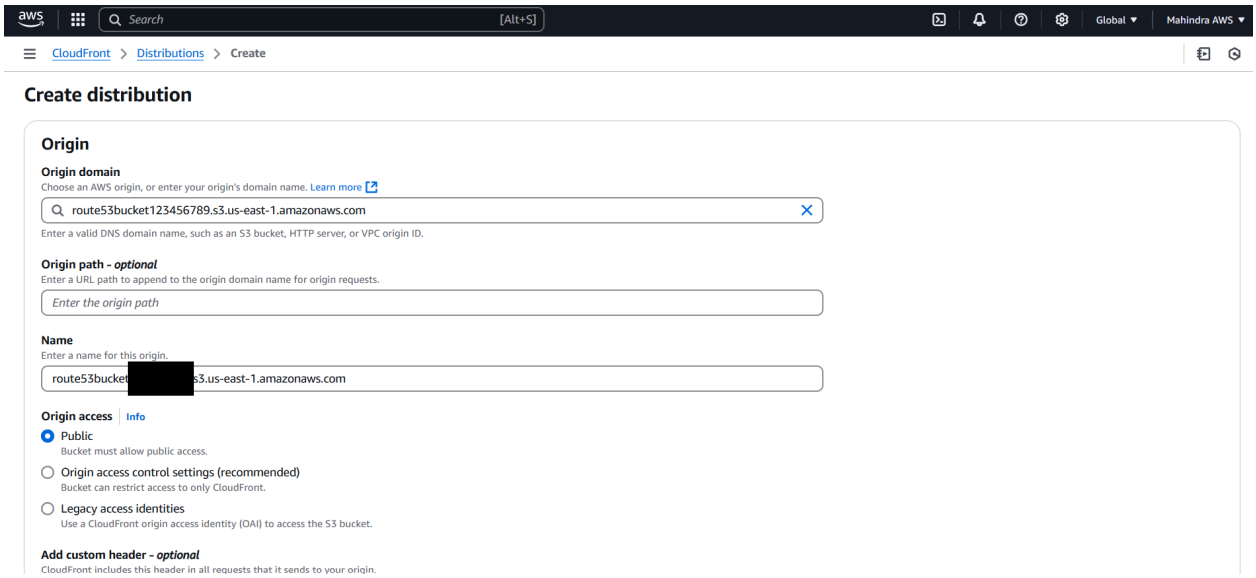
Create a CloudFront Distribution for Your S3-Based Static Website

1. Create a static website in S3 the way you did in Exercise 3.4 in Chapter 3.

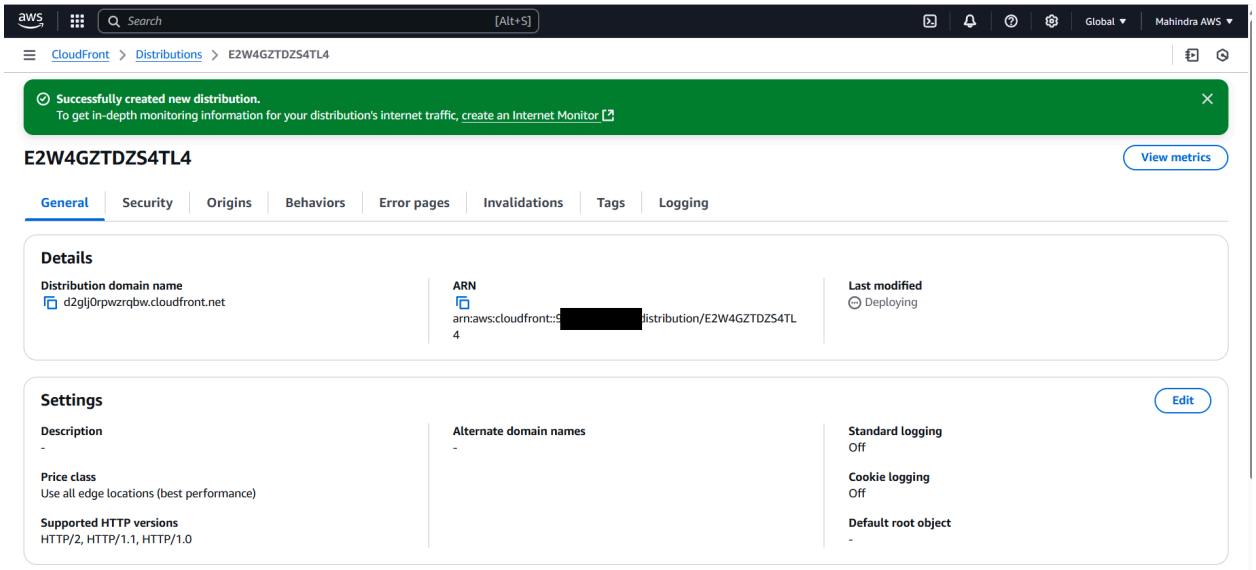

aws
Search
[Alt+S]
United States (N. Virginia)
Mahindra AWS
Amazon S3 > Buckets > Create bucket
Create bucket Info
Buckets are containers for data stored in S3.
General configuration
AWS Region
US East (N. Virginia) us-east-1
Bucket type Info
General purpose
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.
Directory
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.
Bucket name Info
Route53bucket
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). Learn More
Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.
Choose bucket
Format: s3://bucket/prefix
Object Ownership Info
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.
Bucket owner enforced
Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
Block public access to buckets and objects granted through new access control lists (ACLs)
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
Block public access to buckets and objects granted through any access control lists (ACLs)
S3 will ignore all ACLs that grant public access to buckets and objects.
Block public access to buckets and objects granted through new public bucket or access point policies
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
Block public and cross-account access to buckets and objects through any public bucket or access point policies
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.
Turning off block all public access might result in this bucket and the objects within becoming public
AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.
I acknowledge that the current settings might result in this bucket and the objects within becoming public.
Bucket Versioning



2. From the CloudFront dashboard, start a new distribution.
3. Click once inside the Origin domain box, and a list of all the available resources within your account will be displayed. The S3 bucket containing your website files should be included. Select it.

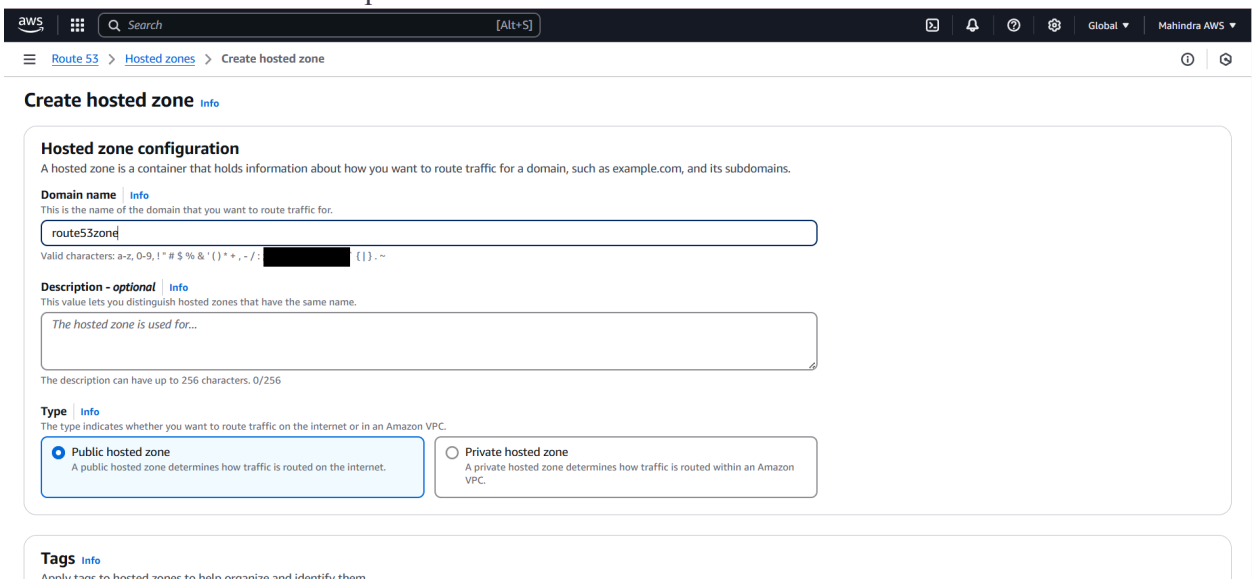


4. Scroll down and select a price class in the Distribution Settings section. Using the All Edge Locations setting will provide the best performance for the greatest number of customers, but it will also cost more. By the way, if you're only creating this as an experiment and you're unlikely to get more than a few hundred requests, then don't worry about the cost; it'll be in the low pennies.

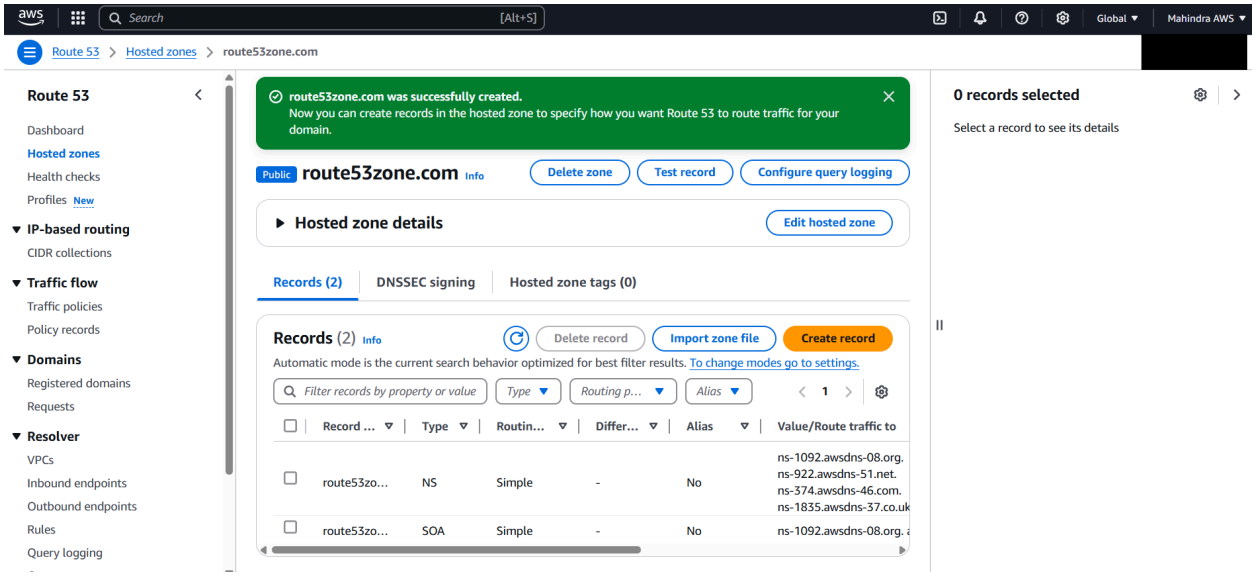


The next three steps will apply only if you have a valid DNS domain to work with.

5. Add the DNS domain names you want to use for this distribution to the Alternate domain name (CNAMES) box. Those might include server1.example.com and www.example.com. If you don't do this, your encryption certificate might not work for all content requests.



6. Click the Request Certificate link beneath the Custom SSL Certificate box and follow the simple instructions to request a certificate for your registered domain name. Once your request has been granted, your certificate will appear in the Custom SSL Certificate box. You can select it.



7. When you're satisfied with your settings, click Create Distribution.
8. In the Route 53 console, create (or edit) a record set that points your domain name to the CloudFront distribution you just created. Wait for the new settings to propagate and confirm that you can access the S3 content through a domain using HTTPS (encryption)

